Blue-Tap: Sub-HCI Bluetooth Vulnerability Discovery, Fuzzing, and Exploitation Framework for Automotive Infotainment

Santhosh Ballikonda
Independent Security Research
santhoshmlaa@gmail.com

***Abstract*—Bluetooth security research lives in three historically-separate tool lineages: standards-level attack proofs of concept, single-layer protocol fuzzers, and below-HCI instrumentation frameworks built from scratch for one chipset family. Practitioners assessing an automotive in-vehicle infotainment (IVI) system must stitch these together by hand, yielding non-reproducible assessments that miss an informationally richer view of the controller side. Even when a host-side probe can detect a CVE by side-effect, host-only tooling cannot read controller RAM or capture LMP. DarkFirmware — originated as a reverse-engineering scaffold by Dark Mentor LLC [1] and extended by us [2] into a 4-hook RTL8761B controller-firmware interception platform with 2,866 RE'd functions and a 16-script below-HCI assessment toolkit — exposes that evidence, turning inferential detection into definitive verification. It is also the *only* path to active exploitation of LMP-resident bugs (KNOB / BIAS / BLUFFS / BrakTooth-class) and to triggering raw-ACL flaws (BlueFrag).**

We present Blue-Tap, a unified Bluetooth pentest framework deployable on commodity USB hardware. Blue-Tap contributes: (1) a 4-hook below-HCI interception platform for the Realtek RTL8761B, released as our open-source DarkFirmware fork [2] that extends the upstream Dark Mentor LLC project [1] with the hook architecture, 6 in-flight modification modes, expanded reverse engineering (slot stride 0x2B8, encryption flag at +0x26, link-key COMB_KEY XOR offset +0x51), and a 16-script assessment toolkit, all exposed to the Blue-Tap module surface via a Python integration layer with post-reset HCI re-resolution; (2) a coverage-guided fuzzer spanning 14 corpus-backed protocols and 16 transport-mapped surfaces with 7 400+ pre-generated seeds, protocol-aware field-weight tracking, response-diversity coverage, and behavioural crash deduplication; (3) 25 CVE detection modules covering 27 unique CVE IDs (2017–2026), 11 non-CVE vulnerability checks, 29 DoS probes (10 CVE-backed), 9 active-exploitation modules, and 7 post-exploitation modules behind structured evidence; and (4) a typed `RunEnvelope` data contract with atomic session persistence yielding bit-identical reports across runs. We compare against BlueToolkit (USENIX WOOT 2025), SweynTooth, and BrakTooth, and exercise the framework against a simulated Bluetooth environment plus a live Android Automotive head unit.

***Index Terms*—Bluetooth security, Link Manager Protocol, coverage-guided fuzzing, firmware instrumentation, automotive security, penetration testing, reproducibility**

I. Introduction

Automotive in-vehicle infotainment (IVI) systems represent an uncommonly exposed Bluetooth attack surface. They run legacy Bluetooth Classic stacks to maintain compatibility with decade-old phones, expose sensitive profiles such as Phonebook Access (PBAP) and Message Access (MAP) as part of normal operation, and are rarely patched after vehicle delivery [3], [4], [5]. A recent large-scale study tested 22 production vehicles, ran 891 tests, and uncovered 128 distinct findings using only host-level tooling [3]; the Tencent Keen Security Lab Tesla campaigns [6] are the canonical real-world automotive Bluetooth case. Yet that result covers only the host-protocol band of the published Bluetooth attack surface; entire bug families — encryption-key negotiation (KNOB [7]), authentication-procedure bypass (BIAS [8]), forward-secrecy breaks (BLUFFS [9]), and the entire Link Manager Protocol (LMP) bug class (BrakTooth [10]) — lives below the Host Controller Interface (HCI) and is unreachable from standard BlueZ sockets.

Prior work has demonstrated that below-HCI visibility is obtainable, but each demonstration builds its instrumentation stack from scratch for one chipset vendor: InternalBlue (Broadcom/Cypress over adb) [11], Frankenstein (Broadcom emulated in QEMU) [12], and the seven ad-hoc patches BLUFFS applies to a reverse-engineered closed-source firmware [9]. None generalise to a commodity USB dongle a pentester can procure in one business day.

In parallel, Bluetooth protocol fuzzers have matured rapidly: SweynTooth [13] and L2Fuzz [14] for BLE / L2CAP, BrakTooth [10] for LMP on ESP32. Each tool finds dozens of CVEs in its niche but does not share seed corpora, crash databases, or report formats with the others, and none produces output that a downstream pentest-report writer can consume without manual reformatting.

Blue-Tap unifies these strands — below-HCI instrumentation, multi-protocol fuzzing, standards-level attack PoCs, and the emerging industry-tooling baseline — into a single framework deliberately scoped for automotive IVI assessment.

a) Contributions.:

1) **DarkFirmware fork — 4-hook below-HCI platform** (Section VII). Building on the initial DarkFirmware project scaffold by Dark Mentor LLC [1], we contribute (a) the 4-hook interception architecture — HCI CMD dispatch (Hook 1), LMP RX with six in-flight modification modes (Hook 2), and outgoing / incoming Link-Controller traffic (Hooks 3–4) — compiled to a sub-1 KB MIPS16e patch; (b) expanded reverse engineering of the RTL8761B (2,866 named functions,

the connection-table slot stride 0x2B8, the encryption-enabled flag at offset +0x26, and the link-key material recovered from `LMP_COMB_KEY` via XOR at +0x51); and (c) a 16-script below-HCI security-assessment toolkit (KNOB probe, BIAS attack, BLUFFS attack, BLURtooth CTKD, LMP and L2CAP fuzzers, encryption inspector, connection-state manipulator, etc.). The fork is released separately [2] so the upstream research community can consume it independently of Blue-Tap; we additionally ship the Python integration layer and the post-reset HCI re-resolution that lets Blue-Tap's modules call these primitives.

2) **Coverage-guided multi-protocol fuzzer** (Section VIII): 7 400+ seeds across 14 corpus-backed protocol generators (SDP, L2CAP, L2CAP-sig, RFCOMM, BNEP, GATT/ATT, SMP, OBEX-PBAP, OBEX-MAP, OBEX-OPP, AT-HFP, AT-phonebk, AT-SMS, AT-inject) plus LMP and Raw-ACL transports for 16 transport-mapped surfaces in total, with a field-weight tracker derived from anomaly and crash ratios, and state-machine selection following AFLNet's inverse-frequency heuristic.
3) **Vulnerability coverage suite** (Section IX): 25 CVE detection modules covering 27 unique CVE IDs (2017–2026), 11 non-CVE vulnerability checks, 29 denial-of-service probes (10 CVE-backed, 19 protocol-stress), 9 active-exploitation modules, and 7 post-exploitation modules, all emitting structured evidence with recovery monitoring.
4) **Typed reproducibility contract** (Section V for the envelope and outcome taxonomy; Section VI for the atomic, `SIGKILL`-safe session store and its crash-consistency theorem): a `RunEnvelope` data format and atomically-written JSON session store that together make every assessment run bit-reproducible at the envelope level.

The rest of the paper is organised as follows. Section II summarises the Bluetooth attack surface and the host/controller boundary. Section III presents the automotive threat model and adversary capability tiers (T1/T2/T3). Section IV positions Blue-Tap against prior tools using those tiers. Section V introduces the framework architecture and the typed envelope contract; Section VI formalises the atomic-write substrate and proves its crash-consistency theorem (contribution (4)). Section VII describes DarkFirmware (contribution (1)), Section VIII the coverage-guided fuzzer (contribution (2)), and Section IX the vulnerability coverage (contribution (3)). Section X evaluates against BlueToolkit, SweynTooth, and BrakTooth. Section XIII discusses limitations and ethics. Section XIV concludes.

II. Background

A. The host/controller boundary

Every Bluetooth implementation is split at the Host Controller Interface (HCI). The *host* side runs on the operating-system CPU and implements L2CAP, RFCOMM, SDP, BNEP, ATT, SMP, and profiles such as OBEX and HFP. The *controller* runs on a dedicated radio SoC and implements the Link Controller (baseband) and Link Manager Protocol (LMP).[1] The HCI is the only interface the host can speak; commands flow downward, events upward, and ACL/SCO data flows bidirectionally.

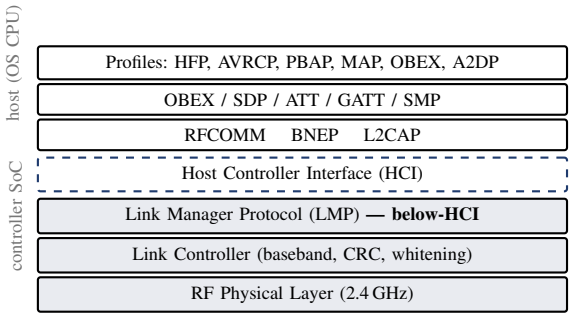

Fig. 1: Bluetooth Classic / BLE stack. Everything above the dashed HCI boundary is implemented by the host operating system; everything below runs in the controller firmware. Blue-Tap's DarkFirmware instrumentation crosses the boundary at the Link Manager level.

Commercial Bluetooth stacks expose no mechanism for the host to observe or transmit LMP. This is deliberate: LMP carries authentication, encryption-key negotiation, and connection-parameter setup, and standards-compliant host stacks cannot override the controller's LMP state machine. The consequence for security research is that *implementation flaws in the controller* (which hosts the Link Controller and Link Manager) are invisible from the host. Five flagship below-HCI bug families (Section X-C) reside in this blind spot: KNOB (CVE-2019-9506), BIAS (CVE-2020-10135), BLUFFS (CVE-2023-24023), the BrakTooth-class LMP fuzzing-discovered bugs, and BlueFrag (CVE-2020-0022) raw-ACL fragmentation.

B. Bluetooth classic and BLE protocol stack

Figure 1 shows the protocol layering. Blue-Tap modules operate at every layer, with the below-HCI layers accessible only through DarkFirmware.

C. Vendor-specific commands (VSC)

HCI reserves Opcode Group Field (OGF) `0x3F` for vendor-specific commands. Most Bluetooth SoCs implement VSCs for firmware update, factory calibration, and debugging. Blue-Tap's DarkFirmware repurposes three VSCs on the RTL8761B (`0xFE22`, `0xFC61`, `0xFC62`) to expose LMP injection and controller memory access to the host (Section VII).

D. Bluetooth security model

Bluetooth pairing has evolved through three generations, each layered on the prior. *Legacy pairing* (BT 1.0–2.0) derives the link key from a numeric PIN via E22/E21 and offers no Man-in-the-Middle (MitM) protection beyond the entropy of the PIN. *Secure Simple Pairing* (SSP, BT 2.1+, [15]) introduces ECDH P-192 key agreement with four association models —

[1] In Bluetooth Low Energy, the equivalent controller-side protocol is the Link Layer (LL).

Numeric Comparison, Just Works, Passkey Entry, and Out-of-Band — selected by the peer-advertised IO Capability bitmap. *Secure Connections* (SC, BT 4.1+) upgrades the elliptic-curve to P-256 and replaces E0/E1 with AES-CCM. The choice between SSP and SC is made during LMP feature negotiation: each side advertises support, and the controller MAY fall back to legacy SSP when either peer lacks SC. This negotiation is the substrate that *KNOB* (low-entropy encryption-key size, [7]), *BIAS* (authentication-procedure downgrade, [8]), and *BLUFFS* (forward/future-secrecy break, [9]) all attack.

Cross-Transport Key Derivation (CTKD, BT 4.2+ [16]) lets a single SSP/SC exchange on Bluetooth Classic seed an LE Long-Term Key (and vice versa), so a successful BR/EDR pairing implicitly grants encrypted LE access. *BLURtooth* (CVE-2020-15802, [17]) abuses this transport bridge to overwrite a strong LE key with a weak BR/EDR-derived one. The broader SMP pairing surface — including IO-capability mismatch, key-length downgrade, and association-model confusion — is surveyed across the SMP literature; formal-verification analyses of Bluetooth pairing [18] provide authoritative descriptions of the protocol invariants that KNOB / BIAS / BLUFFS violate. Blue-Tap models pairing posture explicitly (modules `assessment.pairing_method`, `assessment.pin_lockout`, `assessment.lmp_features`) because automotive deployments routinely retain Just Works fallbacks for backwards compatibility.

E. LMP threat layer

The Link Manager Protocol carries the security-critical handshakes discussed above. Figure 2 groups the 30 most attack-relevant LMP opcodes by security function. KNOB targets `LMP_encryption_key_size_req` (opcode `0x10`) — a single-byte payload field whose value the standard permits to range 1–16; controller firmwares almost never enforce a floor below the peer's request, so an injected 1-byte key downgrade succeeds end-to-end. BIAS targets the authentication procedure `LMP_au_rand`/`LMP_sres` (opcodes `0x0B`, `0x0C`) by replaying or skipping the master-side challenge/response. BLUFFS targets the extended IO-capability and Diffie-Hellman parameter exchange in `LMP_features_req_ext` plus the SC-to-legacy fallback path. The byte-level layout is deliberately minimal — one opcode byte plus 0–16 payload bytes — which is what makes mode-1/mode-4 single-byte modification on DarkFirmware (Section VII) sufficient to mount all three attacks.

F. Automotive IVI attack-surface characteristics

Automotive head units present an unusually exposed Bluetooth surface. They (i) ship with Phonebook Access (PBAP) and Message Access (MAP) profiles always-on so that a paired phone delivers contacts and SMS instantly; (ii) retain pairings for the lifetime of the vehicle, and routinely pair opportunistically with rental or loaner phones [5]; (iii) broadcast a descriptive Class-of-Device that announces the head-unit make and model from across a parking lot; (iv) run BlueZ-derived Linux stacks that are years behind upstream patch cycles — the BlueToolkit [3] large-scale study reported 128 distinct findings across 22 production vehicles, many tied to long-disclosed CVEs that desktop Linux had patched multiple stable releases prior. Android Automotive head units inherit the Android Bluetooth security model and patch-cadence [19]. The combination of always-on profiles, persistent pairings, and slow patch cycles makes IVI systems the natural primary target of Blue-Tap.

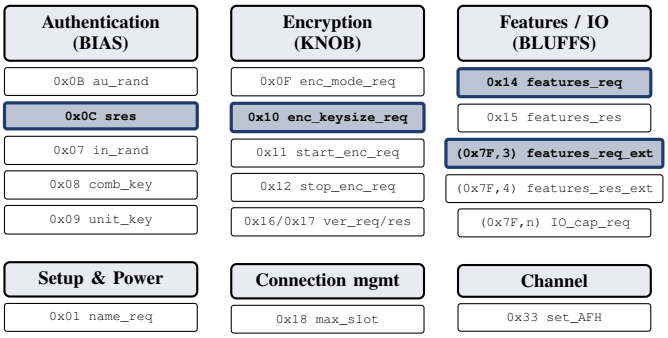

Fig. 2: LMP opcode taxonomy grouped by attack relevance. Filled opcodes (navy) are the LMP packets injected, dropped, or single-byte-modified by the KNOB, BIAS, and BLUFFS exploitation modules in Section VII. The byte-level layout of every LMP packet is one opcode byte plus 0–16 payload bytes; the small field width is what makes single-byte hook-2 modification sufficient.

III. Threat Model

A. Adversary capability hierarchy

We organise Bluetooth adversaries into three capability tiers that correspond to the three tool lineages Blue-Tap unifies. The hierarchy is a strict superset chain: each higher tier subsumes the prior tier's reach plus a new attack-surface band.

a) T1: Passive observer.: Operates within Bluetooth range (∼10 m), able to scan for devices, capture advertising packets, parse SDP, and read public GATT services from any commodity BlueZ-compatible adapter. T1 captures the entire reconnaissance surface and the subset of vulnerabilities exploitable *above* the host stack (*e.g.* unauthenticated GATT writes, SDP traversal). All standard Linux desktops with built-in Bluetooth are T1-capable.

b) T2: Active host injector.: T1 plus the ability to issue arbitrary L2CAP, RFCOMM, BNEP, ATT, SMP, and OBEX packets to a chosen target, optionally after pairing. T2 captures host-protocol-layer attacks: SDP DES-bomb, BNEP heap corruption, unauthenticated-pair flooding, and host-side fuzzing. T2 cannot reliably trigger BlueFrag (CVE-2020-0022) because BlueZ enforces ACL fragmentation correctness on the outbound path. Concurrent work BlueToolkit [3] sits at T2.

c) T3: Below-HCI operator.: T2 plus the ability to observe and inject Link Manager Protocol traffic at the controller boundary. T3 captures KNOB, BIAS, BLUFFS, BlueFrag (raw-ACL confirmation), BrakTooth-class LMP fuzzing, and any attack requiring manipulation of the encryption-key negotiation or authentication-procedure handshakes. Prior T3 tooling

Attacker goal	T1	T2	T3
G1 Reconnaissance	•	•	•
G2 Crypto downgrade			•
G3 Impersonation			•
G4 Data exfiltration		•	•
G5 Denial of service		•	•

Fig. 3: Capability × goal matrix. T1=host passive, T2=host active, T3=below-HCI. •=tier sufficient to achieve the goal; higher tiers inherit. Among the tools we survey (Section IV), Blue-Tap is the only one covering all three tiers from one commodity USB adapter.

(InternalBlue [11], Frankenstein [12], the seven-patch BLUFFS firmware [9]) demands chipset-specific reverse engineering. Blue-Tap's DarkFirmware (Section VII) provides T3 capability on a commodity USB dongle, eliminating the reverse-engineering prerequisite.

Figure 3 cross-tabulates the five attacker goals (G1–G5) against the three capability tiers, marking which tier is strictly required to achieve each goal. The figure makes the Blue-Tap value proposition concrete: one tool, one adapter, all three tiers.

B. Attacker assumptions

a) Attacker.: A Bluetooth-range (∼10 m) adversary with a commodity USB Bluetooth dongle (TP-Link UB500, RTL8761B-based) and a Linux laptop — specifically *not* an SDR, ESP32, custom Broadcom phone, or compromised target CPU. The attacker controls software on her own laptop and dongle, but does not run code on the target. Attack scenarios fall into two pre-conditions: (a) previously paired (*e.g.* the valet opportunity [5]) where the attacker holds the link key; (b) unpaired but in range, where pairing is required as part of the attack itself. Each Blue-Tap module declares its pairing requirement explicitly via the `requires_pairing` descriptor field (Section V).

C. Target assumptions

a) Target.: An automotive IVI head unit implementing Bluetooth Classic 4.0+ and BLE 4.2+. The head unit exposes HFP, PBAP, MAP, A2DP, AVRCP, and one or more vendor-specific SDP records (navigation push, diagnostic AT sub-channels). Firmware patch posture follows industry norm: up-to-date as of vehicle delivery, but rarely refreshed thereafter. The head unit's controller is one of Broadcom, Qualcomm, Mediatek (Airoha), or Realtek; the attack surface enumerated in Section IX is selected to reach detectable vulnerabilities in all four.

D. Attacker goals

We consider five attacker goals, mapped to the Blue-Tap module family that addresses each: (G1) *reconnaissance* — enumerate services, profiles, GATT handles, LMP features (`discovery.*`, `reconnaissance.*`); (G2) *cryptographic downgrade* — force low-entropy keys (KNOB) or session-key reuse (BLUFFS) (`exploitation.knob`, `.bluffs`); (G3) *impersonation* — authentication-procedure downgrade or role swap (`exploitation.bias`, `.encryption_downgrade`); (G4) *data exfiltration* — phonebook, SMS, call audio, file push (`post_exploitation.pbap`, `.map`, `.bluesnarfer`, `.opp`, `.a2dp`); (G5) *denial of service* — crash or deadlock the target Bluetooth subsystem (`exploitation.dos_*`, 29 probes).

a) Out of scope.: Physical attacks on the OBD-II port, CAN bus intrusion, cellular and Wi-Fi attack paths, supply-chain compromise of the head-unit firmware build, and side-channel attacks that require physical instrumentation of the controller SoC. We also do not target unauthenticated head-unit-to-head-unit Bluetooth mesh deployments, which are not part of the IVI Bluetooth threat model.

IV. Related Work

We organise prior work into the three lineages Blue-Tap unifies and one concurrent direct comparable.

a) Below-HCI instrumentation.: InternalBlue [11] is the canonical Broadcom/Cypress firmware-patching framework, reverse-engineered and accessed via adb on Android or directly on Raspberry Pi. It is chipset-specific and requires integration with the target host stack. Frankenstein [12] emulates Broadcom firmware in QEMU and re-hosts it atop Linux for fuzzing; while reproducible, the emulation gap prevents interaction with real radios. BLUFFS [9] applies seven ad-hoc patches to a reverse-engineered closed-source firmware to demonstrate forward- and future-secrecy attacks (CVE-2023-24023). The first three target Broadcom/Cypress. The DarkFirmware project [1] (Dark Mentor LLC, 2025) opened the equivalent reverse-engineering line for the *Realtek* RTL8761B by releasing the project scaffold, Ghidra analysis tooling, and base patch-writer infrastructure. Our fork [2] extends that scaffold into a usable interception platform (4-hook architecture, 6 modification modes, 2,866 RE'd functions, 16-script assessment toolkit) — the substrate on which Blue-Tap's below-HCI capability rests. Blue-Tap then layers the host-side integration, install / recovery state machine, and module surface that makes those primitives usable from every detection, exploitation, and fuzzing module in the framework.

b) Defensive baselines.: The BlueZ project [20] ships the canonical Linux host stack and tooling baseline for our BlueZ tools row in Table XV; BTstack [21] is its embedded counterpart. General-purpose offensive frameworks [22] cover network exploitation broadly but do not natively address Bluetooth radio surface. Boofuzz [23] is the closest network-fuzzing tooling baseline; its lineage informed our protocol-aware mutators (Section VIII-H). Linux kernel SDP / L2CAP CVE clusters [24] drive several of Blue-Tap's 2022 detection modules.

c) Bluetooth protocol fuzzing.: SweynTooth [13] is a state-machine fuzzer for BLE host-peripheral: 12 devices, 11 vulnerabilities, 13 CVEs. BrakTooth [10] attacks the Link Manager of Bluetooth Classic from a dedicated ESP32 radio: 16 vulnerabilities across 13 SoCs. InjectaBLE [25] demonstrates link-layer injection on established BLE connections, described by its authors as the first practical realisation of that attack

class. Classen's post-Frankenstein fitness-tracker work [26] extends Cypress firmware instrumentation to consumer-IoT targets. A recent SoK [27] synthesises a decade of Bluetooth security and privacy research. L2Fuzz [14] is a stateful L2CAP fuzzer achieving 46× more valid malformed packets than prior work. AFLNet [28] is the archetypal code- and state-coverage-guided greybox fuzzer for network protocols. Mirage [29] is an IoT-oriented Metasploit-like framework focused on radio-stack pivoting. Blue-Tap's fuzzer absorbs AFLNet's frequency-aware state selection heuristic (favouring under-explored states, Section VIII-D) and L2Fuzz-style field targeting, while unifying 14 corpus-backed protocols plus LMP and Raw-ACL transports (16 transport-mapped surfaces) behind a single engine.

d) Standards-level attacks (covered detections).: KNOB [7] exploits low-entropy encryption-key negotiation in Bluetooth Classic. BIAS [8] impersonates via authentication-procedure downgrade. BLURtooth [17] abuses cross-transport key derivation (CTKD). BlueBorne [30] is a family of eight BNEP/SDP vulnerabilities in Android / Linux / Windows stacks. All are covered by Blue-Tap detection modules (Section IX); KNOB, BIAS, and BLUFFS additionally have active-exploitation modules that leverage DarkFirmware.

e) Industry tooling baseline.: Commercial automotive cybersecurity products (Cybellum, Karamba XGuard, Argus / Continental, Vector vTESTstudio with CANoe.Bluetooth, Synopsys, Riscure) cover Bluetooth assessment as part of broader vehicle-level security suites; the dominant architectural footprint is static binary / SBOM-driven detection plus host-only protocol probing. Blue-Tap is the *active offensive below-HCI* counterpart to these mostly-static, mostly-host commercial tools rather than a replacement, and its envelope contract (Section V) maps directly onto the evidence requirements of ISO/SAE 21434 §10 (cybersecurity validation) and §15.4 (work-product evidence) and onto the type-approval CSMS evidence that UNECE WP.29 R155 §7.3.4 demands.

f) Concurrent work.: BlueToolkit [3] (USENIX WOOT 2025) is the most directly comparable prior work: a large-scale automotive Bluetooth framework atop BlueZ that ran 891 tests against 22 production vehicles and uncovered 128 findings. BlueToolkit is host-only; it cannot observe or modify LMP. Blue-Tap generalises to the below-HCI attack surface and additionally provides the typed reproducibility contract (Section V) that BlueToolkit lacks.

V. System Architecture

A. Layered design

Blue-Tap is organised into four layers with strict one-directional import rules (Figure 4):

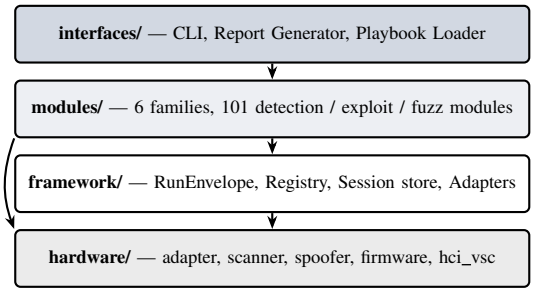

Fig. 4: Four-layer architecture. Arrows denote permitted import direction. `framework/` never imports `modules/`; modules never cross-import between families. This is enforced by CI `ruff` rules and by the test `test_module_registry.py`.

B. Formal envelope contract

Every module returns a `RunEnvelope`, a pure JSON-serialisable dict whose schema is fixed across all 101 modules and 16 transport-mapped fuzzing protocols.

Definition 1 (RunEnvelope)**.** *A Blue-Tap* RunEnvelope *is the 13-tuple*

$$\langle s,\, v,\, m,\, r,\, T,\, a,\, t_0,\, t_1,\, c,\, \sigma,\, E,\, A,\, D\rangle$$

where s is the schema identifier (`blue_tap.<family>.result`), $v = 2$ is the schema version, (m, r, T, a) identify the module, run, target and HCI adapter, (t_0, t_1) are ISO-8601 timestamps, c is the operator context, σ is a module-specific summary, E is a sequence of ExecutionRecords, A is a sequence of ArtifactRefs, and D is the module-specific payload.

Each ExecutionRecord carries two orthogonal status fields:

Definition 2 (Two-field status taxonomy)**.** *An execution has* lifecycle *status $S_\ell \in \{\texttt{completed}, \texttt{failed}, \texttt{error}, \texttt{skipped}, \texttt{timeout}\}$ and* semantic *outcome $S_s \in O_F$, where O_F is the family-specific outcome set for family F. The pair (S_ℓ, S_s) distinguishes "ran cleanly and found nothing" $= (\texttt{completed}, \texttt{not_detected})$ from "crashed before checking" $= (\texttt{error}, \texttt{not_applicable})$.*

Invariant 1 (Module ID)**.** *Every registered module has identifier matching the regular expression `^[a-z0-9_]+(\.[a-z0-9_]+)+$` and the first dotted segment equals the module family. The identifier is mandatory on every envelope; a missing or malformed id raises `ValueError` at envelope-construction time, before any data reaches disk.*

The atomic session store that persists each `RunEnvelope` to disk is the load-bearing reproducibility primitive and is given its own treatment in Section VI; the relevant identifiers referenced from later sections are Algorithm 2 (the `SIGKILL`-safe write), Theorem 1 (its consistency guarantee), and Algorithm 3 (the per-command append flow).

C. Module lifecycle

A Blue-Tap module is any subclass of `framework.module.base.Module`. The class declares a frozen `ModuleDescriptor` containing seven required fields — `module_id`, `family`, `name`, `description`, `protocols`, `requires`, `entry_point` — plus optional fields for `schema_prefix`, `destructive`,

Input : module-id m, target T, adapter a, params p, session S
Output : persisted RunEnvelope file path

$d \leftarrow$ `registry.try_get`(m)
if $d = \perp$ **then raise** `ModuleNotFound`
`check_prereqs`$(d$.`requires`$)$ `// HCI / pairing / DarkFirmware`
$cls \leftarrow$ `importlib.import_module`$(d$.`entry_point`$)$ `// lazy class load`
$obj \leftarrow cls(adapter = a)$
emit `run_started` CLI event (m, T)
$E \leftarrow$ `builder.start`$(d, T, a, S$.`operator_ctx`$)$
try
  $E \leftarrow obj$.`run`(T, p, E)
on exception ϵ
  E.`execution_status` $\leftarrow$ `error`;
  E.`error` $\leftarrow \epsilon$;
  emit `run_error`
E.`completed_at` $\leftarrow$ `utcnow()`
`validate_envelope`(E) `// schema-prefix & outcome`
return `Session.log_command`(S, E) `// atomic; alg. 2`

Algorithm 1: End-to-end module invocation pipeline

`requires_pairing`, and `report_adapter_path`. Two registration paths exist:

1) *Explicit:* the family's `__init__.py` calls `get_registry().register(d)` on each descriptor at import time. Used by reconnaissance, exploitation, post-exploitation, and fuzzing modules.
2) *Auto-registration via `__init__subclass__`:* the assessment family contains 25 CVE detection modules, each a class under `modules/assessment/checks/`. The base `Module` class implements `__init__subclass__(cls)` to call `get_registry().register` on the subclass's `descriptor()` method automatically. Importing the family package transitively imports every `cve_*.py` file, populating the registry with no per-module boilerplate.

Algorithm 1 shows the resulting end-to-end invocation pipeline, from operator command to envelope persistence. The pipeline is strictly synchronous and single-threaded; concurrent assessments use multiple session directories rather than thread pools, preserving the atomicity argument of Theorem 1.

D. Master architecture overview

Figure 5 shows every Blue-Tap subsystem and the data-flow paths between them. The master figure complements the strict layering of Figure 4 by exposing the runtime graph: which modules import which envelope builders, which adapters match which schemas, and where the atomic-write boundary sits.

TABLE I
Bundled playbooks. Risk levels follow the `interfaces/cli` convention.

Playbook	Phase scope	Risk
`passive-recon`	discovery + recon	none
`quick-recon`	discovery + recon (fast)	low
`full-assessment`	vulnscan + recon + LMP capture	low
`lmp-fuzzing`	DarkFirmware + LMP fuzzing campaign	medium
`ivi-attack`	vulnscan + KNOB/BLUFFS + hijack	**high**

TABLE II
Built-in report adapters and the envelope schemas they consume. Schemas use the prefix `blue_tap.<family>.`; only the suffix is shown.

Adapter	Schema suffix(es)	Section produced
Discovery	`discovery.scan`	Targets list
Recon	`recon.{sdp,gatt,...}`	Service / GATT trees
Vulnscan	`assessment.result`	CVE / V-check findings
Attack	`exploitation.result`	Exploit narrative
DoS	`exploitation.dos.result`	DoS probe + recovery
Fuzz	`fuzzing.campaign`	Crash table + corpus
LMP Capture	`firmware.lmp_capture`	LMP packet log
Firmware	`firmware.install`	Hook activation status
Data	`post_ex.{pbap,map,...}`	Phonebook / SMS dumps
Audio	`post_ex.{a2dp,hfp}`	Audio session log
Spoof	`hardware.spoof`	BDADDR / CoD spoof

E. Playbook composition

Operators rarely run modules in isolation. Blue-Tap ships five YAML playbooks (Table I) that compose modules into phase-ordered DAGs and bind module parameters to playbook-level variables. Figure 6 shows the data dependencies in the `ivi-attack` playbook — the most complete chain — which threads vulnerability scan output into exploitation parameters and finally into a report.

F. Report adapters

Eleven built-in report adapters consume envelopes and emit `SectionModel` trees for the HTML and JSON renderers. Adapters are matched to envelopes by `schema` prefix; dispatch is a sorted predicate scan,

$$dispatch(e) \;=\; \texttt{head}\big(\texttt{sort}_{prio}\big[\, a \mid a \in Adapters,\; a.\texttt{accepts}(e) \,\big]\big), \tag{1}$$

with plugin adapters at default priority 50 and built-in adapters at 100. Lower priority sorts first: a plugin adapter that `accepts(e)` is given the chance to render *ahead* of any built-in adapter for the same schema, so plugin renderings take precedence and built-ins serve as the fallback for envelopes no plugin claims — the inverse of an overlay-style "layer over." Table II lists the 11 adapters and the schemas they consume.

Adding a new module type is a single-file change: implement `ReportAdapter`, register in `adapters/__init__.py`. Registration also accepts a plugin-supplied `report_adapter_path`, used by the entry-point plugin mechanism (Section V-C) so third-party modules ship their own renderers without forking the core.

G. Runtime CLI events

Every operator-visible side effect of a module is emitted as a typed CLI event by

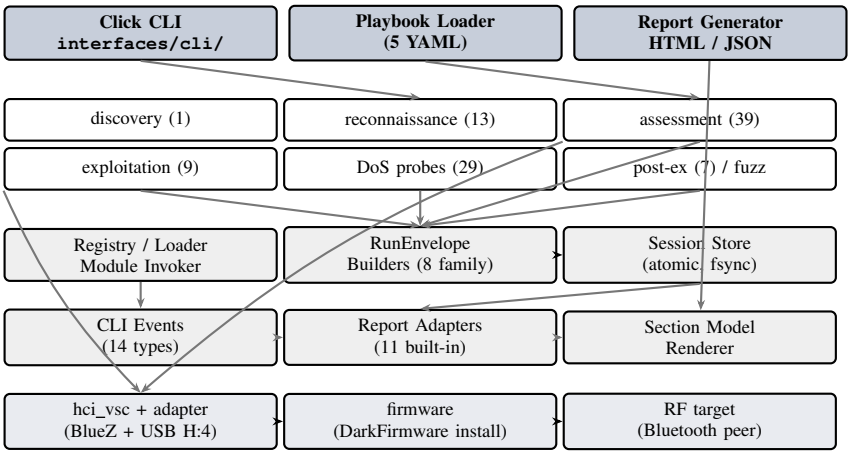

Fig. 5: Blue-Tap master architecture. Top: operator-facing interfaces. Second row: the six module families (101 modules total). Third row: the framework's runtime contracts — registry, envelope builders, atomic session store. Fourth row: CLI events, the 11 report adapters, and the section/renderer pipeline. Bottom: hardware plumbing (BlueZ HCI, DarkFirmware install path, RF). The black arrows denote envelope flow; gray arrows denote control or event flow.

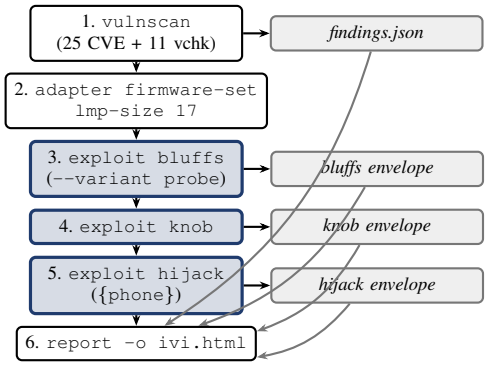

Fig. 6: Execution graph for the `ivi-attack` playbook. Navy nodes require DarkFirmware. Italic boxes are envelope artifacts piped into the final `report` step. Parameter binding substitutes {`phone`} from the command line into the `exploit hijack` step.

TABLE III
Canonical CLI event types. Categories denote semantic role of the event in operator dashboards and structured-log consumers.

Category	Event type	When
Run lifecycle	`run_started`	module begins
	`run_completed`	module ends, OK
	`run_aborted`	operator → Ctrl-C
	`run_error`	unhandled exception
Phase	`phase_started`	sub-step begins
Execution	`execution_started`	probe sent
	`execution_result`	response classified
	`execution_skipped`	precondition not met
	`execution_observation`	passive log
Pairing	`pairing_required`	module needs link key
Recovery	`recovery_wait_started`	post-DoS wait
	`recovery_wait_progress`	periodic update
	`recovery_wait_finished`	target alive again
Artifact	`artifact_saved`	file persisted

`framework/runtime/cli_events.py`. Fourteen event types are canonical (Table III); any other string is allowed but logged as a warning to keep operator dashboards forward-compatible while flagging contract drift in CI.

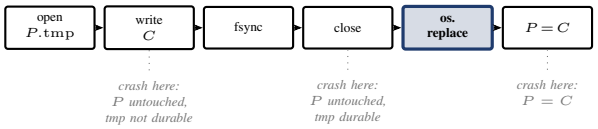

Fig. 7: Atomic-write timeline (Algorithm 2). Only the accent step (os.replace) mutates P; any earlier crash or `SIGKILL` leaves P in its pre-call state. The three crash annotations match the three case splits in the proof of Theorem 1; on the `O_TMPFILE` path, a kill before the post-rename fsync leaves no orphan tempfile because the kernel reaps the unnamed inode on process death.

VI. Reproducibility Substrate

The framework's reproducibility claim (contribution (4) of Section I) rests on a single primitive: every `RunEnvelope` (Definition 1) reaches disk through an atomic, `SIGKILL`-safe write that admits no torn-write, no orphan tempfile, and no ambiguity about what was logged at the time of a crash. This section formalises the primitive (Algorithm 2, Theorem 1), delineates its filesystem-scope assumptions, and specifies the per-command append flow (Algorithm 3) that `Session.log_command` invokes from every `Module.run` return path.

A. Atomic envelope write

Blue-Tap persists every `RunEnvelope` to `sessions/<name>/NNN_cmd.json` via the atomic-write pipeline of Algorithm 2. The pipeline branches on filesystem capability: Linux with `O_TMPFILE` support takes the no-debris path (anonymous inode reaped on process death); other platforms fall through to a PID-suffixed named tempfile that preserves atomicity but admits a bounded, attribution-clean fragment in the event of a kill.

Figure 7 traces the on-disk state at each step and marks the three crash windows that the consistency proof case-splits.

Theorem 1 (Crash consistency, no debris)**.** *For any system-crash or `SIGKILL` point during Algorithm 2, the post-recovery directory state satisfies both: (i) P contains its pre-call content*

```
Input  : file path P, parent dir D, serialised envelope C
Output : P contains C or its pre-call content; no orphan tempfile
if Linux ∧ O_TMPFILE supported on D then
    f ← open(D, O_TMPFILE)      // anonymous inode
    write(f, C); fsync(f)
    T ← P ‖ ".tmp." ‖ pid
    linkat(f, D, T)      // materialise: ∼μs window
    close(f)
else
    T ← P ‖ ".tmp." ‖ pid        // per-PID: no clobber
    f ← open(T); write(f, C)
    fsync(f); close(f)
end
os.replace(T, P)     // atomic POSIX rename
fsync(D)     // commit rename to journal
```

Algorithm 2: Atomic envelope write (SIGKILL-safe)

or C; never a torn, partial, or coexistent write; and (ii) on the `O_TMPFILE` path, no orphan tempfile remains under D. The fallback path admits at most one $P \| $".tmp."$\| pid$ fragment per crashed writer, distinct from any concurrent writer's fragment by construction.

Proof sketch. Atomicity (i): Before `fsync`, the tempfile is unflushed and P is untouched. Between `fsync` and `os.replace`, the tempfile is durable and P is still untouched. `os.replace` is an atomic POSIX rename: the kernel either commits the new mapping or aborts; no intermediate state is visible. After `os.replace`, P holds C. *No debris (ii)*: On the `O_TMPFILE` path the tempfile is an unnamed inode with refcount $= 1$; the kernel reaps it on process death until `linkat` materialises a name. The materialisation window spans only `linkat`–`os.replace`, on the order of microseconds. On the fallback path the tempfile name is PID-suffixed, so a stale fragment is uniquely attributable to the crashed writer and cannot collide with any concurrent writer's tempfile. □ □

B. Filesystem scope

The theorem assumes POSIX rename semantics with crash-consistent metadata, which holds on ext4-default and btrfs-default. The `O_TMPFILE` path additionally requires kernel support for unnamed-inode materialisation via `/proc/self/fd`, available on mainline Linux since 3.11; on filesystems that refuse `O_TMPFILE` or `linkat` from `/proc` (9p, some FUSE, some NFS) the implementation falls back to the PID-suffixed tempfile path, which preserves atomicity but admits the bounded debris of Theorem 1(ii). The parent-directory `fsync` after `os.replace` guarantees the rename reaches the journal before any subsequent envelope is appended; without it the guarantee degrades on ext4 with `data=writeback`. Network filesystems (NFS, SMB) inherit the underlying server's atomic-rename semantics, which may be weaker than POSIX; session directories on such mounts are out of scope. This crash-consistency property is what lets a crashed assessment be resumed from the last envelope without ambiguity about what was logged.

```
Input : session dir S, envelope e, command name c
n ← session_meta(S).command_count + 1
P ← S ‖ sprintf("%03d_%s.json", n, c)
C ← JSON-encode(e)
invoke Algorithm 2 with (P, S, C)      // atomic, SIGKILL-safe
if success then
    increment command_count;
    persist session_meta via Algorithm 2
    // also atomic
end
return P
```

Algorithm 3: Atomic envelope-append (`Session.log_command`)

C. Per-command append flow

Algorithm 3 formalises `Session.log_command` — the operation that every `Module.run` return invokes — showing the per-command sequence number that drives the `NNN_cmd.json` naming and the meta-file refresh that follows on each successful append. The session-meta file is itself written through Algorithm 2, so a crash partway through an append leaves either both the per-command JSON and its meta entry persisted, or neither — the same atomic guarantee that holds at the file level extends to the meta-data update, which is critical for the resume-after-crash semantics of Section X-D.

VII. DarkFirmware: A 4-Hook Below-HCI Platform on Commodity Hardware

Blue-Tap's below-HCI mechanism is the DarkFirmware fork [2] we maintain at https://github.com/Indspl0it/DarkFirmware_real_i, building on the initial DarkFirmware reverse-engineering project by Dark Mentor LLC [1]. The upstream contribution is the project scaffold: the Ghidra analysis tooling, the repository layout, the initial RTL8761B decompilation, and the base patch-writer infrastructure. Our fork extends it into a usable below-HCI interception platform: the 4-hook architecture and 6 in-flight modification modes (Tables IV and VI), a sub-1 KB MIPS16e patch (∼600 B in the in-tree binary), expanded reverse engineering covering 2,866 named functions[2] and the controller's connection-table memory layout (Table VII), and a 16-script below-HCI security-assessment toolkit. Blue-Tap then layers a host-side Python integration on top: the atomic install pipeline (Algorithm 4), the post-reset HCI re-resolution (Section VII-K),

[2]Function-name inventory exported from the fork's pyghidra analysis pass; commit `50830180` of [2].

and the module surface (Section V-C) that exposes LMP injection and controller-memory R/W to every assessment, exploitation, and fuzzing module. We chose the RTL8761B as the substrate because it ships in a widely available commodity adapter (TP-Link UB500), uses a MIPS16e core patchable via vendor-specific HCI commands, and exposes a sufficiently flexible command-dispatcher hook point.

A. Reverse-engineering methodology

The four hook addresses (Table IV: `0x80133FFC`, `0x80133FF8`, `0x80133FF4`, `0x80133FEC`) were located by static analysis of the stock RTL8761B firmware disassembled with Ghidra (MIPS16e processor module via `pyghidra`) plus selective dynamic confirmation through VSC `0xFC61` memory-read scans. Three landmarks anchored the analysis: the HCI-command dispatcher (located by following the USB-bulk-OUT-endpoint handler), the LMP RX/TX entry points (located by following the `tLC_RX` interrupt vector referenced from the controller's task table at `0x800F_0000` band), and the connection-table base `0x8012DC50` (located by cross-referencing the public `HCI_CREATE_CONNECTION` handler against post-pairing RAM deltas). The expanded `pyghidra`-driven analysis named 2,866 functions in the ROM + patch region and decompiled the LMP-critical primitives — `send_LMP_reply`, `LMP_ENCRYPTION_KEY_SIZE_REQ`, `LMP_AU_RAND`, `LMP_COMB_KEY`, `patch_replaces_assoc_w_tLC_TX`; from those decompilations we derived the connection-slot stride `0x2B8`, the encryption-enabled flag offset `+0x26`, and the link-key material that the controller XORs into `LMP_COMB_KEY` at `+0x51`. Each hook's backup-address slot is a 4-byte controller-RAM word inside the firmware's unused initialised-data region (verified zero-valued in stock firmware before patch; non-zero after). The host checks "DarkFirmware active" through a two-stage probe rather than a single slot read. *Stage 1 (backup-slot probe)*: VSC `0xFC61` reads Hook 1's backup at `0x80133FFC`; non-zero data implies the hook installer ran and the controller is patched. *Stage 2 (LMP-TX echo probe)*: if Stage 1 reads zero (early boot, slot not yet materialised, or a partial install), the host issues VSC `0xFE22` (LMP TX) with a 3-byte dummy payload; DarkFirmware mirrors the payload back as an HCI vendor event (`0xFF`) before the command-complete, while stock Realtek firmware emits only the command-complete with status `0x01`. Stage 2 guards against a class of false-negatives where the slot read returns 0 despite the firmware being patched. The MIPS16e patch was authored on top of the upstream patch-writer scaffold and is bit-identical between the standalone fork release and the binary that Blue-Tap installs; binary diff of the in-tree `rtl8761bu_fw_darkfirmware.bin` against the unpatched original yields ∼600 bytes of changes (520 B appended, 81 B in-place). The methodology is portable to other Realtek BT chipsets that share the command-dispatcher entry-point shape (RTL8761BU sibling, RTL8821C, RTL8822CU); a port to Broadcom or Mediatek would be redundant because InternalBlue [11] already covers Broadcom.

TABLE IV
DarkFirmware hooks. Addresses are MIPS16e RAM locations in the RTL8761B controller; `file.py:line` references `blue_tap/hardware/firmware.py`.

Hook	Intercepts	Backup addr	Purpose
H1	HCI CMD dispatch	`0x80133FFC`	VSC 0xFE22 → LMP TX
H2	LMP RX	`0x80133FF8`	Log + 6 mod modes
H3	`tLC_TX`	`0x80133FF4`	Log outgoing LMP/ACL
H4	`tLC_RX`	`0x80133FEC`	Log incoming LC frames

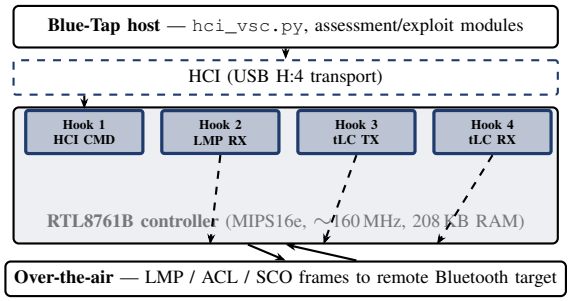

Fig. 8: DarkFirmware architecture. Blue-accent nodes are injected hooks; dashed arrows are vendor event `0xFF` log streams flowing back to the host. The HCI boundary (dashed) remains standards-compliant; existing hosts see only additional vendor events.

B. Four-hook architecture

DarkFirmware installs four hooks into the controller's 208 KB RAM region (Table IV, Figure 8). Each hook has a *backup address* that `firmware.py` reads via VSC `0xFC61` to verify the hook is active; a non-zero value at `0x80133FFC` (Hook 1's backup slot) is the canonical active-DarkFirmware indicator.

The four hooks are not symmetric in their persistence. Hooks 1 and 2 are baked into the patched firmware binary itself, so they survive every USB reset and replug at the cost of nothing more than reloading `rtl8761bu_fw_darkfirmware.bin`. Hooks 3 and 4 (the `tLC_TX` / `tLC_RX` link-controller intercepts) have their shim code in the binary too, but the controller's boot sequence reinitialises the `tLC_TX`/`tLC_RX` function pointers from ROM *after* the patch loads, so the shims are dormant until the host writes their backup pointers into RAM. Activation is two VSC `0xFC62` memory writes (`HOOK3_BACKUP_ADDR`, `HOOK4_BACKUP_ADDR`) followed by a read-back verify — <10 ms total — but the operational consequence is that Hooks 3 and 4 must be re-armed after every USB reset, and a long-running campaign that triggers a reset (Section VII-J) loses LC-frame visibility until `init_hooks()` runs again. Hook 1's backup slot is the single canonical indicator because it is the only one of the four whose presence in RAM is sufficient to certify the install pipeline ran end-to-end.

C. Vendor-specific command interface

Three VSCs carry all host↔controller below-HCI traffic, with payload layouts shown in Table V. The maximum LMP injection payload is patched from the stock 10 bytes to 17

TABLE V

VSC and vendor-event interface used by DarkFirmware. The first three rows are HCI vendor-specific commands (OGF `0x3F`); the last row is HCI Event `0xFF`, the standards-defined vendor-event opcode that carries hook output back to the host.

Code	Direction	Purpose
`0xFE22` (VSC)	H→C	LMP injection (*17 B* payload)
`0xFC61` (VSC)	H→C	4-byte controller memory read
`0xFC62` (VSC)	H→C	1–4 byte controller memory write
`0xFF` (Event)	C→H	Vendor event carrying hook output

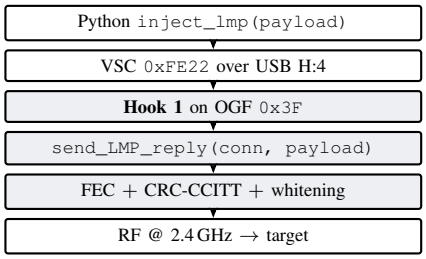

Fig. 9: LMP-injection call chain. Only Hook 1 (navy) is DarkFirmware. The crypto/CRC/whitening stages are untouched, preserving on-air compliance.

bytes by writing to RAM offset `0x8011167E` at firmware-load time.

D. Hook-2 modification modes

Hook 2 (incoming LMP) supports six modes selected by writing to `MOD_FLAG_ADDR` = `0x80133FF0`; the configuration word is packed as

$$\begin{aligned} W &= (\text{offset} \,\&\, \texttt{0xFF}) \\ &\mid ((\text{value} \,\&\, \texttt{0xFF}) \ll 8) \\ &\mid ((\text{opcode} \,\&\, \texttt{0xFF}) \ll 16). \end{aligned}$$

Modes 0–5 offer passthrough logging, one-shot byte modification, one-shot drop, persistent opcode drop, persistent byte modification, and auto-respond, respectively. The persistent-modify mode (4) is how Blue-Tap mounts KNOB: every `LMP_encryption_key_size_req` (opcode `0x10`) is modified to request 1 byte of entropy, across as many rounds as the target initiates.

E. LMP injection path

Figure 9 traces a host-originated LMP packet. Critically, the controller's FEC, 16-bit CRC-CCITT, and LFSR whitening remain in the transmit path — injected LMP frames are indistinguishable from legitimate ones at the physical layer. This is what makes KNOB, BIAS, and BLUFFS standards-compliant when mounted from a Blue-Tap adapter.

F. Patching and post-reset re-resolution

Installation is a six-step sequence: backup stock firmware, atomic install of the patched binary, USB reset, HCI re-enumeration, hook activation via `0xFC62`, and verification by reading all four backup addresses. Step 4 — HCI re-enumeration — is the non-obvious engineering challenge.

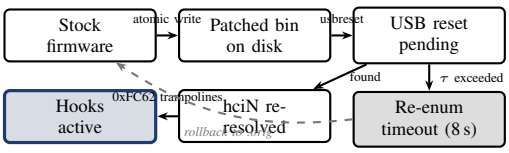

Fig. 10: DarkFirmware install state machine. Solid transitions are the success path; the dashed transition is the operator-visible failure recovery (atomic-install rollback). The accent state is the target steady state (Algorithm 4).

After `usbreset`, the Linux kernel often assigns a *different* `hciN` index; a naive implementation continues talking to the old index and silently fails. Blue-Tap resolves the re-enumerated index via a three-tier discovery cascade that tolerates both currently supported RTL8761B dongle variants — the TP-Link UB500 (`2357:0604`) and the generic Realtek RTL8761BU (`0BDA:8771`) — without the operator specifying which is plugged in. *Tier 1 (`lsusb` presence)*: `lsusb -d <VID:PID>` is run against each entry of the supported-VID:PID tuple; absence of any match is a fast-fail before any sysfs walk. *Tier 2 (sysfs modalias)*: for every `/sys/class/bluetooth/hci*` whose `device` symlink resolves to a USB device, the modalias is read and matched against `v2357p0604` or `v0bdap8771`; the first hit yields the HCI name. *Tier 3 (`hciconfig` manufacturer fallback)*: on kernels where the HCI's immediate parent does not expose a modalias readable by the host, `hciconfig -a` is parsed and the manufacturer string *Realtek* or its IEEE company-id rendering `(93)` is matched. Once an HCI index is bound, the host blocks until `hciconfig` reports `UP RUNNING` with a non-zero BDADDR — firmware-load completion can lag the sysfs entry by several seconds, and VSCs issued in that window bind to stale controller state. The `RTL8761B_VID_PIDS` tuple in `hardware/firmware.py` is the single point of extension: adding a new RTL8761B-bearing dongle is a one-line change, and the modalias matcher absorbs it without operator-visible changes. Median end-to-end install latency is ∼7 s on the bench setup; the design upper bound (Section VII-G) is ∼10 s.

Algorithm 4 formalises the install/re-enumeration sequence and Figure 10 shows the corresponding state diagram. The bound on the `usb_reset_and_wait` loop ensures `firmware.py` surfaces a clean error rather than blocking operator commands indefinitely on a faulty dongle.

G. Hook installation timing bounds

The pipeline of Algorithm 4 is bounded at design time rather than empirically tuned. The post-reset kernel-cleanup sleep is fixed at 1.5 s; the HCI re-enumeration polling loop is bounded by an 8 s timeout (poll interval 0.5 s); the four hook trampoline writes complete in tens of VSC round trips on USB H:4. The end-to-end install therefore completes within an upper bound of ∼10 s, dominated by the kernel-cleanup sleep that we cannot eliminate. The atomic-install boundary precludes corrupted-firmware failure modes (Theorem 1): any failure on the new firmware write leaves the original `.orig` backup untouched, and any hook-write failure surfaces a clean operator-visible exception rather than a half-installed state.

Input : patched firmware F_p, USB id $v{:}p$, timeout τ=8 s
Output : active hci index h, or $\perp$

```
cp rtl8761bu_fw.bin{,.orig}   // backup (/lib/firmware/)
atomic-install F_p via Algorithm 2   // atomic, SIGKILL-safe
usbreset(v:p); sleep 1.5 s   // kernel cleanup
t ← 0
while t < τ do
    scan /sys/class/bluetooth/hci*/modalias for v:p
    if match h then
        foreach hook ∈ {H1, H2, H3, H4} do
            b ← VSC 0xFC61 read at hook.backup_addr
            if b = 0 then write hook trampoline via VSC 0xFC62
        end
        return h
    end
    sleep 0.5 s;  t ← t + 0.5
end
return ⊥   // operator-visible failure
```

Algorithm 4: DarkFirmware install with post-reset HCI re-resolution

H. LMP opcode coverage table

Hook 2 distinguishes *observation* (mode 0, log only) from *modification* (modes 1–5). Table VI enumerates the LMP opcodes Hook 2 can manipulate, organised by attack class. The key observation is that single-byte modification of opcode 0x10 suffices for KNOB, single-byte modification of feature_req_ext parameters suffices for BLUFFS, and `auto-respond` (mode 5) on opcode 0x0B is sufficient for BIAS — no opcode-format-specific parser is required in firmware, just a (offset, value, opcode) triple.

Figure 11 renders the six Hook 2 modification modes as a state machine. Modes 1 and 2 are auto-clearing (single-shot); modes 3–5 persist until the operator explicitly resets the `MOD_FLAG_ADDR` word. KNOB exploitation uses mode 4 (persistent modify) because LMP key-size negotiation can be re-attempted by either peer mid-session and the operator has no means of predicting how many rounds will fire.

I. Memory R/W performance and connection-table inspection

VSC `0xFC61`/`0xFC62` expose 4-byte aligned reads and writes to controller RAM, with sub-word writes via mode bits encoding the byte count. Round-trip latency is dominated by USB H:4 transfer scheduling, which is host-stack dependent. The qualitative implication is that scanning the entire 208 KB RAM region requires roughly 52 k 4-byte reads — usable for offline forensic inspection but too slow for inline use during an active exploit. The operator-facing primitives therefore target specific slot fields (Table VII) rather than performing whole-region scans.

For the inline use case — KNOB key-size verification — Blue-Tap reads a single connection-table slot field. The RTL8761B keeps a 12-slot table at base `0x8012DC50`, each slot `0x2B8` bytes. The fields relevant to security checks are summarised in Table VII. The `key_size` read at `slot_base + 0x23` is the canonical evidence that a KNOB downgrade succeeded: the controller actually negotiated a 1-byte key, not just acknowledged the LMP request.

J. BDADDR spoofing cascade

The same VSC `0xFC62` memory-write primitive that activates hooks doubles as the substrate for BDADDR spoofing, with two operationally distinct paths whose tradeoffs the operator selects implicitly via the cascade of Algorithm 5. Path A (*RAM-only patch*) writes the new BDADDR directly to every copy in controller memory and returns within milliseconds, with no USB reset; the change is volatile and reverts on the next replug or power cycle. Path B (*firmware-file patch + USB reset*) rewrites the BDADDR field in

TABLE VI
LMP opcodes addressable from Hook 2. "Mod" = modes 1–5 can change bytes; "Obs" = mode 0 logs only. All opcodes are observable; modification reach is determined by payload-format constancy, not firmware parser logic.

Opcode	Mnemonic	Reach	Attack
`0x07`	`in_rand`	Mod	pairing-bootstrap
`0x0B`	`au_rand`	Obs / Drop	BIAS (suppress)
`0x0C`	`sres`	Obs	BIAS verify
`0x0F`	`enc_mode_req`	Mod	enc-downgrade
`0x10`	`enc_keysize_req`	Mod(1/4)	**KNOB**
`0x11`	`start_enc_req`	Obs	timing
`0x12`	`stop_enc_req`	Obs	passive
`0x14`	`features_req`	Mod	SSP/SC profile
`0x15`	`features_res`	Mod	feature spoof
`(0x7F,3)`	`features_req_ext`	Mod	**BLUFFS**
`(0x7F,4)`	`features_res_ext`	Mod	BLUFFS verify
`0x16/17`	`version_req/res`	Obs	passive
`0x33`	`set_AFH`	Obs	passive

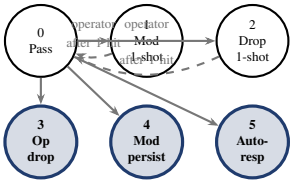

Fig. 11: Hook 2 modification modes. Solid filled (navy) modes are persistent; dashed transitions are auto-clear after one packet match. Operator selects mode by writing (offset | value≪8 | opcode≪16) to `MOD_FLAG_ADDR`.

TABLE VII
RTL8761B connection-table slot layout (offsets from slot base; 12 slots at `0x8012DC50` base, stride `0x2B8`).

Offset	Field	Used for
`+0x00`	remote BD_ADDR (6 B)	target binding
`+0x23`	encryption key size (1 B)	**KNOB verification**
`+0x26`	encryption-enabled flag	SC posture
`+0x214`	secure-connections flag	BIAS / BLUFFS posture

Input : adapter a, target BDADDR b
Output : spoof verdict $\in \{\texttt{verified}, \texttt{stale}, \texttt{failed}\}$

if $\neg$ *is_darkfirmware_loaded*(a) **then**
  return `failed`
end
Path A (RAM): `patch_bdaddr_ram`(a, b) via VSC `0xFC62`
$b' \leftarrow$ `hciconfig.read`(a)
if $b' = b$ **then return** `verified` (RAM)
Path B (firmware file): rewrite BDADDR field in `*.bin`
atomic-install via Algorithm 2; `usbreset`
re-resolve $a' \leftarrow$ `find_rtl8761b_hci()`
$b'' \leftarrow$ `hciconfig.read`(a')
if $b'' = b$ **then return** `verified` (file)
return `stale`

Algorithm 5: BDADDR spoofing on a DarkFirmware adapter

`rtl8761bu_fw_darkfirmware.bin`, atomically installs the patched blob (Algorithm 2), and triggers a `usbreset` that re-runs the install pipeline of Algorithm 4; the change is persistent across replug but pays the ∼7 s post-reset re-resolution latency. The cascade attempts Path A first and falls through to Path B only when `hciconfig` verification reports a stale address after the RAM patch — a condition we observed when host-stack caches lag the controller-side BDADDR by an unbounded interval. Path A covers the common case of one-shot impersonation during a paired session; Path B covers long-running campaigns where the operator needs the spoofed identity to survive controller resets or kernel re-enumeration. The cascade is the same shape we use elsewhere (adapter discovery, DarkFirmware active-check): a fast happy-path followed by a more expensive but more robust fallback.

K. Failure modes and rollback

Three failure classes are explicitly handled. (i) *Atomic-install abort*: write to `$F_p.tmp` fails; the original `rtl8761bu_fw.bin` is never replaced (Theorem 1); the operator sees an exception and the dongle stays on stock firmware. (ii) *HCI re-enumeration timeout* (Algorithm 4 returns $\perp$): observed occasionally on heavily-loaded USB hosts; the operator is instructed to retry. `firmware.py` keeps the prior `.orig` backup, so the dongle remains usable as a stock controller. (iii) *Hook-write VSC failure*: a single hook trampoline write returning non-zero status; `firmware.py` treats this as fatal and refuses to mark DarkFirmware active. The capability detector (Section VII-L) therefore reports stock mode, and modules requiring DarkFirmware skip cleanly with `module_outcome = not_applicable` rather than crash.

Rollback to stock firmware is a one-step operation: restore the `.orig` backup and `usbreset`; no controller state is held outside the firmware blob.

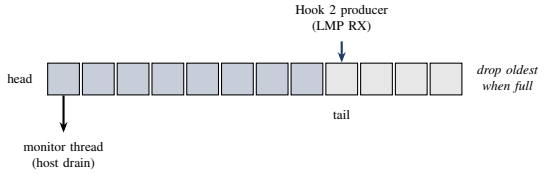

Fig. 12: Vendor-event ring buffer (host-side, per hook). Filled slots (navy) are unread events between head (slot 0) and tail (slot 8); gray slots are free. The producer (Hook 2) writes at the tail; the host monitor drains from the head. The drain is single-threaded and lock-free w.r.t. `recv_event` (Section V-G); under overload the producer overwrites the oldest unread slot deterministically rather than blocking the controller. The single-threaded invariant is enforced at the socket layer: `HCIVSCSocket.recv_event()` raises `RuntimeError` on cross-thread access while the monitor loop holds the socket, preventing event-frame corruption from interleaved `recv` calls on a single HCI fd.

L. Vendor-event ring buffer

Hooks 2–4 emit structured log records as HCI vendor event `0xFF`, up to 256 B per event. The host maintains a bounded `collections.deque(maxlen=1000)` per hook to absorb bursts. Loss is intrinsic when controller event rates exceed the USB drain capacity; the ring buffer drops oldest events deterministically rather than blocking the controller. Figure 12 shows the buffer geometry. Sequence-numbered records are proposed future work that would let the analyser detect drops directly; today the host infers loss from gap detection in LMP transaction IDs.

M. Below-HCI attack surface opened

Per the coverage matrix `paper/data/cve-matrix.csv`, the host-only detection probes for the 27 CVE IDs Blue-Tap covers all run from a BlueZ socket; below-HCI is required only for the *active confirmation* of CVE-2020-0022 (the BlueFrag DoS probe in `exploitation/dos/checks_raw_acl.py`, which injects a crafted raw-ACL fragment that BlueZ's outbound fragmenter would otherwise mask). The detection-only paths are deliberately host-reachable so operators without a DarkFirmware-flashable dongle still get full detection coverage. Three of the nine active-exploitation modules (KNOB, BIAS, BLUFFS) require DarkFirmware for their injection path (Sections IX-B to IX-D); the remaining six active exploits and all detection probes function without it.

VIII. Coverage-Guided Multi-Protocol Fuzzer

Blue-Tap's fuzzer is structured around a single `FuzzCampaign` engine that orchestrates 14 corpus-backed protocol generators plus LMP and Raw-ACL transports (16 transport-mapped surfaces in total) with shared transport, mutation, crash-database, and health-monitoring subsystems.

A. Engine and campaign orchestration

The campaign loop is round-robin over protocols: each iteration selects a protocol, loads a seed, applies a strategy-chosen mutation, sends via the appropriate transport, classifies the response, and updates per-protocol statistics. Three

TABLE VIII
Mutation operators. All operate post-field-selection on the chosen byte range.

Operator	Behaviour
`bitflip`	flip one or n consecutive bits
`byte_insert`	insert 1–n random bytes
`byte_delete`	delete 1–n consecutive bytes
`byte_replace`	overwrite with random byte(s)
`chunk_duplicate`	duplicate a random chunk
`chunk_shuffle`	permute chunks
`truncate`	drop a suffix
`extend`	append random bytes
`IntegerMutator`	boundaries {0, 1, 0x7F, 0x80, 0xFF, ...}
`LengthMutator`	zero, minimal, off-by-one over/under, max

stopping conditions are supported: total wall-clock budget, total iteration budget, and `SIGINT`. Key constants are `DEFAULT_COOLDOWN_SECONDS` = 10.0 (default inter-iteration pause), `TARGET_DEAD_TIMEOUT` = 30.0 s, and the dual stats trigger `STATS_INTERVAL_SECONDS` = 60.0 / `STATS_INTERVAL_ITERATIONS` = 1 000.

B. Protocol-aware field-weight mutator

For each (protocol, opcode) pair Blue-Tap maintains a static *ProtocolFieldMap* describing the byte ranges of named fields. The `FieldAwareMutator` weights field selection by empirical productivity. Let a_f be the cumulative count of anomalous responses attributed to mutations of field f, let c_f be the cumulative crashes, and let n_f be the number of mutations attempted on f. The per-field weight is

$$w_f \;=\; 1.0 \,+\, \alpha\,\frac{a_f}{\max(n_f,1)} \,+\, \beta\,\frac{c_f}{\max(n_f,1)}, \qquad (2)$$

with $\alpha = 5.0$ (`ANOMALY_BOOST`) and $\beta = 20.0$ (`CRASH_BOOST`). Weights are re-normalised to sum to one every `DEFAULT_UPDATE_INTERVAL` = 100 mutations per protocol:

$$p_f \;=\; \frac{w_f}{\sum_{f'\in F} w_{f'}},$$

yielding a categorical distribution $\Pr(\text{select } f) = p_f$. The constants α and β are design choices, not derived from a published bound: the 4:1 crash-to-anomaly ratio reflects the intuition that fields whose mutations crash the target deserve a larger budget boost than fields that merely produce structural anomalies. Closely related field-prioritisation ideas appear in BrakTooth [10] and L2Fuzz [14]; the specific weights are tunable through the `ANOMALY_BOOST` and `CRASH_BOOST` constants and operators may re-tune for their own targets.

C. Mutation catalogue

Table VIII summarises the eight byte-level and two specialised mutators. Havoc mode applies 5–20 random operations per case, following AFL++ convention [31].

D. State-machine selection

Many Bluetooth protocols are stateful. Blue-Tap models protocol state with a `StateID` tuple $\langle P, o, e, x\rangle$ (protocol, opcode, error code, extra) and prioritises under-explored states with an AFLNet-inspired inverse-frequency score

$$score(s) \;=\; \frac{1+\lambda(s)}{1+f(s)}, \qquad (3)$$

where $f(s)$ is the observed frequency of state s across the campaign and $\lambda(s) \geq 0$ is a discovery-recency bonus favouring recently-discovered states. The 1 added in the denominator is Laplace-style smoothing: a freshly-discovered state ($f(s) = 0$) yields a finite, bounded score rather than the ∞ a naive $1/f(s)$ would produce, which would otherwise saturate the categorical-distribution sampling. The exact form differs from the AFLNet weighted-frequency score [28] but captures the same intuition; AFLFast's Markov-chain formulation [32] is the closest published analogue. For LMP specifically, we additionally encode the 16 BrakTooth-class targets as state-transition preconditions; reaching any such state fast-tracks that state's fuzzing budget.

E. Crash detection and deduplication

The response analyser classifies each transaction into one of {`NORMAL`, `STRUCTURAL`, `LENGTH_MISMATCH`, `TIMING`, `SIZE_DEVIATION`, `UNEXPECTED_OPCODE`, `LEAK_INDICATOR`, `BEHAVIORAL`} and flags a `CrashType` via a Z-score threshold on latency and length:

$$anomaly(\ell, n) \;=\; \left[\tfrac{|\ell-\mu_\ell|}{\sigma_\ell} > 3\right] \,\vee\, \left[\tfrac{|n-\mu_n|}{\sigma_n} > 3\right], \qquad (4)$$

applied per-opcode to latency ℓ and response-length n; the four baseline parameters $(\mu_\ell, \sigma_\ell, \mu_n, \sigma_n)$ are learnt from the first 100–500 transactions per opcode. Either dimension exceeding the threshold flags the transaction as a crash candidate. Crashes deduplicate by *behavioural signature*

$$sig = \mathrm{SHA256}\big(type \,\|\, resp[:32] \,\|\, \mathrm{ascii}(|resp|)\big), \qquad (5)$$

where $\mathrm{ascii}(\cdot)$ denotes the decimal-string encoding of the response length (matching the Python reference implementation: `str(len(resp)).encode()`). Pinning the length-field encoding this way ensures that two implementations in different languages produce bit-identical signatures from the same response. The signature is payload-independent: different mutations triggering the same underlying bug produce identical signatures and collapse to one entry. The 32-byte response prefix is sized for the typical short Bluetooth-error responses (ATT errors are 5 B, L2CAP command-rejects are 4 B); two semantically distinct bugs producing the same short error response will over-merge into one signature. Operators can tighten the signature by including the `StateID` tuple at the cost of larger-than-necessary crash-database growth; the framework defaults trade dedup aggressiveness for false-merge tolerance.

F. Minimisation

Algorithm 6 shows the delta-debugging (ddmin [33]) reducer, one of three available strategies (binary search, ddmin, field-level). In practice, a 48-byte crash input reduces to ∼6 bytes in

Input : crash input c, reproducer predicate rep
Output : 1-minimal input $c^\star$

$n \leftarrow 2$
while $|c| \geq 2$ **do**
  partition c into n subsets $c_1, \ldots, c_n$
  foreach c_i **do**
    if $rep(c \setminus c_i)$ **then**
      $c \leftarrow c \setminus c_i$; $n \leftarrow \max(n-1, 2)$; **continue**
    end
  end
  if $n \geq |c|$ **then**
    return c
  end
  $n \leftarrow \min(2n, |c|)$
end
return c

Algorithm 6: Delta-debugging crash minimiser (ddmin)

TABLE IX
Seed counts per protocol, measured from the in-tree generators by enumerating each `_PROTOCOL_GENERATORS` entry in `blue_tap/modules/fuzzing/corpus.py`. LMP is supplied via DarkFirmware rather than a host-side generator and is listed separately.

Protocol	Seeds	Protocol	Seeds
SDP	858	OBEX-PBAP	1,096
L2CAP	166	OBEX-MAP	1,082
L2CAP-sig	17	OBEX-OPP	1,068
RFCOMM	239	AT-HFP	65
BNEP	580	AT-phonebk	38
BLE-ATT	411	AT-SMS	38
BLE-SMP	650	AT-inject	78
LMP (DarkFirmware)	1,111		
Total			**7,497**

50–200 tests; each test respects a per-cooldown-window retry budget to distinguish true minimisers from flaky reproducers.

G. Seed corpus

Table IX lists the 14 corpus-backed protocol generators with their measured seed counts; the LMP transport (carried by DarkFirmware rather than a host-side generator) is shown separately for context. The total across all 15 rows is 7 497 seeds. Seeds are sampled with anomaly-weighted probability $\Pr(i) = w_i / \sum_j w_j$, where the unnormalised weight is

$$w_i \;=\; 1 \;+\; 2\,a_i, \tag{6}$$

and a_i is the cumulative anomaly count for seed i; this focuses budget on seeds empirically associated with surprising responses.

H. Mutation operator selection

A single fuzz iteration selects (i) a protocol π, (ii) a seed s from the protocol corpus weighted by Eq. (6), (iii) a field f within s weighted by Eq. (2), and finally (iv) a mutation operator op from the catalogue of Table VIII. The choice of op is conditional on the field's declared type in the `ProtocolFieldMap`: integer-typed fields draw from `IntegerMutator`'s boundary set $\{0, 1, 0\text{x}7\text{F}, 0\text{x}80, 0\text{xFF}, \ldots\}$; length-typed fields draw from `LengthMutator` (zero, off-by-one, max); generic byte-range fields use the eight `FieldMutator` byte-level operators. Havoc mode (drawn with low probability per iteration; default 0.05) applies 5–20 random operators back-to-back as in AFL++ [31]; in greybox-fuzzing literature havoc mode disproportionately produces crashes relative to its iteration share [31], [32].

TABLE X
Per-protocol `StateID` extractors and what each field encodes. *Bucket* columns map continuous values (PSMs, ATT handles) to coarse equivalence classes: PSM bucket $\in$ {fixed BR/EDR, dynamic, vendor}; handle bucket $\in$ {system-info, primary-svc, characteristic, vendor}. The campaign visits each bucket at least once before fine-tuning within it, so $f(s)$ in Eq. (3) stays meaningful even on targets with thousands of distinct handles.

Protocol	o (opcode)	e (error)	x (extra)
SDP	PDU_ID	error code	continuation flag
L2CAP	sig opcode	cmd-reject code	PSM bucket
ATT	ATT opcode	ATT err code	handle bucket
SMP	SMP opcode	SMP err code	assoc model
RFCOMM	DLCI MUX cmd	rsp code	channel
LMP	LMP opcode	not-accepted	feature bit

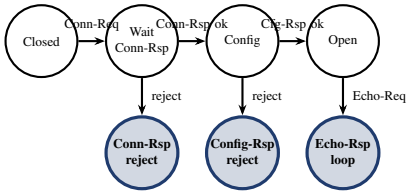

Fig. 13: Worked example: L2CAP signalling-channel FSM as encoded by `extract_state_l2cap`. Filled (navy) states have low historical visit frequency and therefore high inverse-frequency score under Eq. (3); the campaign loop prioritises seeds whose response classification reaches them.

I. Per-protocol state-machine encodings

The `StateID` tuple $\langle P, o, e, x \rangle$ (Section VIII-D) is materialised differently for each protocol family by family-specific extractors in `state_inference.py`. Figure 13 shows the worked-out L2CAP signalling-channel FSM, the largest and most attack-rewarding example. Other extractor schemas are summarised in Table X. The under-explored states (low $f(s)$ in Eq. (3)) are the ones the campaign loop prioritises; in our L2CAP campaigns these are predominantly the post-error-response states, which are reached only by a small fraction of seeds and historically harbour bugs (e.g. CVE-2022-20345, CVE-2022-42895).

J. Crash triage pipeline

When the response analyser flags an iteration as a crash candidate the pipeline of Figure 14 kicks in: behavioural signature (Eq. (5)) is computed, the `crash_db` is queried for an existing match, and on a miss the minimiser (Algorithm 6) reduces the input. Cluster identity is the $(target, \pi, \sigma)$ triple, so different mutations that trigger the same underlying defect collapse to a single row in the operator report rather than

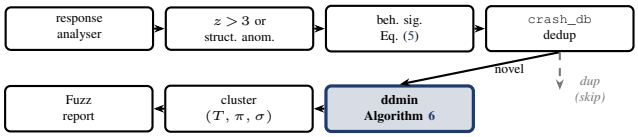

Fig. 14: Crash-triage pipeline. Duplicate signatures bypass the minimiser (the dashed exit) and feed cluster counts directly. Novel signatures cycle through ddmin reduction before clustering, so the report contains 1-minimal payloads only.

appearing as independent crashes. The dedup matters because raw analyser output on any non-trivial target is dominated by structural-anomaly noise (length-field mismatches, opcode echoes, timing jitter) that resolves to the same signature across many mutations; the SHA-256 collapse converts operator triage from per-anomaly to per-signature. Absolute campaign yields depend on target hardness and time budget — comparable BR/EDR campaigns against production-class targets report on the order of 11–16 distinct findings over multi-month windows [10], [13], which sets the realistic expectation for Blue-Tap deployments rather than orders-of-magnitude larger numbers.

K. Campaign scheduling

The default scheduler is round-robin over the protocol set Π. Each protocol receives an equal slice per outer iteration; within a slice the field-weight tracker (Section VIII-B) drives field selection. We optionally re-weight the protocol slice itself by recent crash density: if c_π is the crash count in the last 1 000 iterations of protocol π, the next iteration draws π from

$$\Pr(\pi) \;\propto\; 1 + \gamma \, \frac{c_\pi}{\max\!\big(\max_{\pi' \in \Pi} c_{\pi'},\, 1\big)}, \tag{7}$$

with $\gamma = 2.0$ by default. This is a soft multi-armed bandit over the protocol set, complementary to Eq. (2) (which biases field selection *within* a protocol). The framing is inspired by UCB-style multi-armed-bandit exploration [34] and by AFLFast [32]'s Markov-chain model of coverage-guided fuzzing, but Eq. (7) is not a published bandit algorithm and we do not claim a regret bound; the scheduler is a deliberately simple soft-prioritisation heuristic. Stopping conditions are wall-clock budget, iteration budget, or operator interrupt; Algorithm 7 formalises the resulting main loop.

L. 0-day discovery workflow

Figure 15 traces the operator workflow from a fresh campaign to a CERT-disclosure-ready advisory. Concretely: a novel behavioural signature in step 3 is the analyser's signal that the target's response was unlike any observed pattern; if the signature reproduces under an isolated retry (cooldown + repair), it advances to ddmin (step 4). The minimised payload plus the captured response is the artifact required by CERT/CC for intake. Blue-Tap ships the `CrashDB.export_json` helper that bundles the 4-tuple $(\tilde{s}^\star, r, sig, \text{StateID})$ into a disclosure JSON keyed by reproduction recipe. The 0-day claim of Blue-Tap is methodological — the framework provides the pipeline; we do not claim discovered 0-days in this paper, and any such discoveries from future campaigns will be disclosed to vendors prior to publication.

Input : protocols Π, budget B (time $\vee$ iterations), target T
Output : CrashDB $\mathcal{C}$, corpus updates Δ

initialise $\mathcal{C}, \Delta$; learn baselines per $\pi \in \Pi$
while *budget remains* $\wedge$ `health_alive`(T) **do**
  $\pi \leftarrow$ scheduler-draw via Eq. (7)
  $s \leftarrow$ seed $\sim \text{Multinom}(w_1, \ldots, w_N)$ `// Eq. (6)`
  $f \leftarrow$ field $\sim \text{Multinom}(p_1, \ldots, p_K)$ `// Eq. (2)`
  $op \leftarrow$ mutator(f.`type`); $\tilde{s} \leftarrow op(s, f)$
  send $\tilde{s}$ via transport$_\pi$; measure latency ℓ, response r
  $z \leftarrow$ Z-score$(\ell, |r|)$ vs. baseline `// Eq. (4)`
  if $z > 3 \vee$ `structural_anomaly`(r) **then**
    $\sigma \leftarrow \text{SHA256}(type \| r[:32] \| |r|)$ `// Eq. (5)`
    if $\sigma \notin \mathcal{C}$ **then**
      $\tilde{s}^\star \leftarrow$ `ddmin`$(\tilde{s})$ `// Algorithm 6`
      $\mathcal{C} \leftarrow \mathcal{C} \cup \{(\sigma, \tilde{s}^\star, r, t)\}$; update field weights via Eq. (2)
    end
  end
  if *iteration* $\bmod 100 = 0$ **then**
    renormalise p_f; update state-scores via Eq. (3)
  end
end
return $\mathcal{C}, \Delta$

Algorithm 7: Coverage-guided campaign main loop

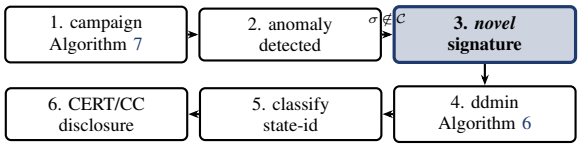

Fig. 15: 0-day discovery workflow. The accent step is the behavioural-signature novelty check (Eq. (5)); the remaining steps are mechanically driven by the campaign loop and triage pipeline.

IX. Vulnerability Coverage

Blue-Tap's assessment family ships 25 CVE detection modules covering 27 unique CVE IDs across 2017–2026, 11 non-CVE vulnerability checks (legacy-pairing, encryption-enforcement, exposure, capability, and authorisation gates), and 29 denial-of-service probes (10 CVE-backed, 19 protocol-stress). Figure 16 plots the CVE distribution by disclosure year; Table XI gives the per-protocol breakdown across detection, vulnerability check, and DoS probe families.

Per `paper/data/cve-matrix.csv`, one of the 27 unique CVE IDs covered (CVE-2020-0022 BlueFrag) requires DarkFirmware for the detection probe itself; the remaining 26 are detectable from a host-only adapter. The three flagship standards-level attacks — KNOB (CVE-2019-9506), BIAS (CVE-2020-10135), and BLUFFS (CVE-2023-24023) — are

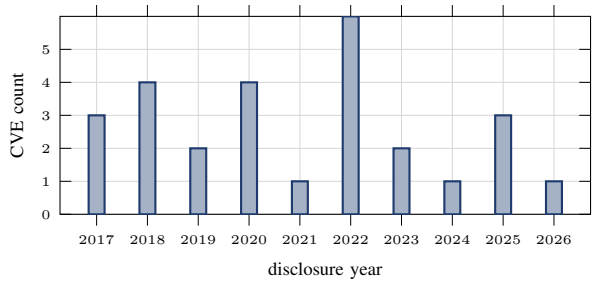

Fig. 16: Distribution of the 27 unique CVE IDs covered by Blue-Tap's 25 CVE detection modules, grouped by disclosure year. Peak year 2022 reflects BrakTooth disclosure spillover into mainline BlueZ; 2025–2026 captures Mediatek Airoha (RACE) and upstream kernel patches.

TABLE XI

Vulnerability coverage summary by protocol family. The CVE column counts unique CVE IDs covered by detection modules (`cve_*` files in `modules/assessment/checks/`); some modules cover multiple CVE IDs, so the row count (27) exceeds the module count (25). Vulnerability-check (V-check) and DoS columns are module counts. Auditable via `paper/data/cve-matrix.csv`, `dos-matrix.csv`, and `vchecks.csv` (one row per (module, CVE-id) pair).

Protocol	CVE IDs	V-check	DoS
L2CAP	9	0	4
SMP	5	0	2
RFCOMM	1	4	2
BNEP	2	0	2
SDP	1	0	3
GATT / ATT / EATT	3	2	1
HID	2	0	0
ACL (raw)	1	0	1
AVRCP / AVDTP	1	0	2
OBEX	0	1	1
HFP / AT	0	0	3
LMP	0	1	6
RACE (Airoha)	1	0	0
Classic (misc)	1	3	2
Total	**27**	**11**	**29**

implemented as both detection checks *and* active-exploitation modules, with the latter exercising the full LMP-injection path of Section VII.

A. Detection-module anatomy

Every CVE detection module follows the same five-phase shape (Algorithm 8, Figure 17): *discover* the relevant target capability via SDP / GATT / LMP feature exchange; *elicit* the suspicious behaviour by sending the smallest payload the CVE description specifies; *observe* the response or side-effect (response-code, latency, link state, controller-RAM field); *classify* the observation into the family-canonical outcome set (`confirmed`, `not_detected`, `inconclusive`, `pairing_required`, `not_applicable`); and *evidence* the result via the `EvidenceRecord` (summary, packets, responses, capability limitations). The shape is not enforced by an interface — it is a discipline encoded in the family's `ModuleDescriptor.description` field — but every existing module conforms to it. The `capability_limitations` field of the evidence record

Input : target T, adapter a, descriptor d (CVE id, indicators)
Output : `ExecutionRecord` with `module_outcome` $\in O_{asm}$ (Definition 2)

$cap \leftarrow$ `discover_capability`(T, d) `// SDP / GATT / LMP feat`
if $\neg cap$.`applicable` **then**
  return (`not_applicable`, cap.`reason`)
end
if d.`requires_pairing` $\wedge \neg paired(T)$ **then**
  return (`pairing_required`, $\perp$)
end
$P \leftarrow$ `build_probe`$(d$.`indicators`$)$
$r \leftarrow$ `send_and_recv`(P, T, a) `// transport-aware`
$o \leftarrow$ `observe`(r, T) `// e.g. key_size at slot+0x23`
$c \leftarrow$ `classify`$(o, d$.`indicators`$)$
evidence $\leftarrow$ `EvidenceRecord`$(P, r, o, c, caps)$
return $(c$, `evidence`$)$

Algorithm 8: Generic CVE detection probe (assessment family)

Fig. 17: Five-phase shape of every CVE detection module. Observation (navy) is the phase that distinguishes Blue-Tap's host-only checks from its DarkFirmware-augmented checks: only DarkFirmware lets the observe phase read controller RAM or capture LMP packets.

is what lets the report adapter say "probed but unreachable from the host" rather than mis-reporting a clean negative.

B. Case study A — KNOB (CVE-2019-9506)

KNOB exploits Bluetooth Classic's encryption-key-size negotiation: the standard permits keys of 1–16 bytes and the controller is expected (but not required) to enforce a configured floor. A Blue-Tap KNOB exploitation runs as three phases: (i) *probe* from the host: read peer LMP features and confirm the target advertises encryption support; (ii) *negotiate*: under DarkFirmware, configure Hook 2 in mode 4 (persistent modify, Section VII-H) on opcode `0x10`, offset 1, value `0x01`; (iii) *verify*: read the controller-side key_size field at `slot_base + 0x23` (Table VII). If the field reads `0x01` after the negotiation completes, the target accepted a 1-byte key.

Optionally, the KNOB module includes an *E0 brute-force validator*: knowing the entropy is 8 bits, the host can derive the session key from a single observed encrypted frame. Blue-Tap ships a real E0 implementation (`exploitation/_e0.py`, not a mock); a 1-byte key is exhausted in $\sim$10 ms on a single CPU. Figure 18 traces the LMP exchange.

C. Case study B — BIAS (CVE-2020-10135)

BIAS [8] demonstrates that an attacker who knows only the victim's BD_ADDR — and who does *not* hold the long-term link key — can still impersonate the victim. Antonioli et al.

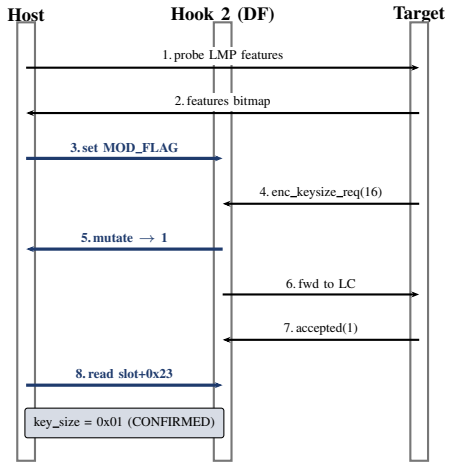

Fig. 18: KNOB exploitation swimlane. Steps 3, 5, and 8 (navy, bold) are DarkFirmware-mediated. Step 3 configures Hook 2 with (opcode $= 0x10$, offset $= 1$, value $= 0x01$, mode $= 4$); step 5 mutates the inbound `LMP_enc_keysize_req(16)` to keysize 1; step 8 reads controller RAM at `0x8012DC50+0x23` and finds key_size $= 0x01$, confirming the downgrade end-to-end.

Input : target T, DarkFirmware adapter a, target slot i
Output : `ExecutionRecord`; `module_outcome` $\in \{$`success`, `recovered`, `not_applicable`$\}$

Phase 1 — probe: $F \leftarrow$ LMP feature mask via host-side recon
if $\neg F$.`has_encryption` **then return** `not_applicable`
Phase 2 — negotiate:
`set_mod_flag`(a) with opcode$=$`0x10`, offset$=1$, value$=$`0x01`, mode$=4$
trigger encryption start (e.g. pairing or auth refresh on T)
wait for LMP_accepted vendor event
Phase 3 — verify: $k \leftarrow$ `vsc_read`$(a, \texttt{0x8012DC50} + i \cdot \texttt{0x2B8} + \texttt{0x23})$
if $k =$ `0x01` **then return** `success`
else if $k >$ `0x01` $\wedge$ $k <$ `0x10` **then return** `recovered (partial)`
else return `not_detected`

Algorithm 9: KNOB active exploitation (Phases 1–3)

exploit two protocol gaps: (i) Secure Connections (SC) silently falls back to legacy authentication when either peer claims SC is unsupported, and (ii) legacy authentication is *unilateral* after a master/slave role switch, so a master that issues no challenge to its slave is accepted as authenticated by the slave's controller without verification. Blue-Tap's detection variant probes the four indicators that gate the attack: SSP enabled?, SC enabled end-to-end on this connection?, BT version ≥ 4.1?, auto-reconnect to previously-paired peer? The active variant uses DarkFirmware Hook 2 in mode 3 (Section VII-H) to *drop* the incoming `LMP_au_rand` (opcode `0x0B`) on the master side after a role-switch, suppressing the standards-mandated

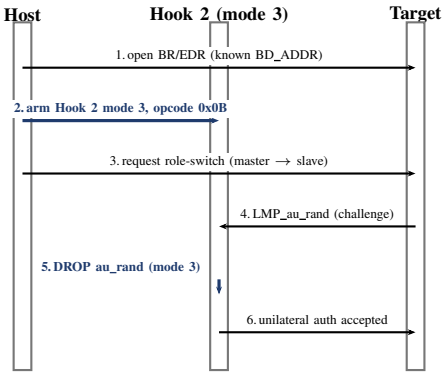

Fig. 19: BIAS impersonation swimlane. The mechanism is the *absence* of a mutual-authentication challenge after role-switch, not a forged `LMP_sres`. Step 5 (navy) drops the responder's `LMP_au_rand` on the attacker side; the responder accepts the unilateral authentication path because the spec permits it. No link-key knowledge is required.

mutual-auth challenge; the slave's controller then accepts the unilateral authentication path and the attacker is bonded as the victim's BD_ADDR. The implementation does *not* compute or pre-load `LMP_sres`: BIAS does not require knowledge of the link key. Figure 19 sketches the flow. The exploit is "standards-compliant" on-air because DarkFirmware does not bypass the link-controller's CRC and whitening path (Section VII).

D. Case study C — BLUFFS (CVE-2023-24023)

BLUFFS [9] breaks Bluetooth Classic's *forward and future secrecy* guarantees by exploiting the under-specified diversifier negotiation in the SSP / Secure Connections session-key derivation function (KDF). The attacker forces the controller to derive multiple sessions' keys from a predictable diversifier, so a session key recovered for one session can be re-derived for another — breaking forward secrecy of past sessions and future secrecy of subsequent ones. Blue-Tap's implementation follows the original disclosure [9] closely but ports it to a commodity Realtek adapter. The four-phase exploitation: (i) observe the `LMP_features_req_ext` (escape opcode `(0x7F,3)`) and the subsequent `LMP_start_encryption_req` (`0x11`) at the next encryption start; (ii) Hook 2 mode 1 modifies the diversifier-relevant payload bytes in the encryption-mode / features-extended exchange so the responder's KDF input has attacker-controlled low entropy; (iii) the controller derives a session key that the attacker can re-derive offline from the known diversifier and a prior session-key recovery; (iv) the attacker can then decrypt past or forward future ciphertext on the same paired link. The Blue-Tap module declares `requires_pairing=true` because BLUFFS targets an already-paired session; the operator must therefore have either a prior pairing window or one of the BIAS / KNOB exploits to bootstrap. Figure 20 shows the schematic.

E. DoS taxonomy and recovery monitoring

The 29 DoS probes split as in Table XII: 10 CVE-backed and 19 protocol-stress. Every probe is followed by a

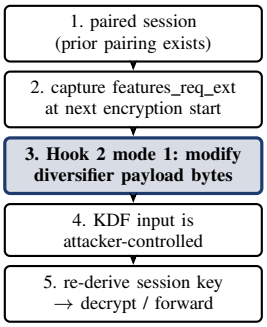

Fig. 20: BLUFFS forward-secrecy break. Step 3 (navy) is the only DarkFirmware-mediated step; the rest is captured by the host-side analyser.

TABLE XII
29 DoS probes. CVE-backed probes (top) include the upstream advisory; protocol-stress probes (bottom) are crafted from fuzzer-corpus reduction.

Probe	Protocol	Severity	Recovery
CVE-backed (10)			
`cve_2017_0781_bnep_heap`	BNEP	HIGH	l2ping
`cve_2017_0782_bnep_underflow`	BNEP	HIGH	l2ping
`cve_2019_19192_att_deadlock`	BLE ATT	MED	BLE-adv
`cve_2019_19196_key_size`	BLE SMP	MED	BLE-adv
`cve_2020_0022_bluefrag`	ACL	CRIT	dev-name
`cve_2022_39177_avdtp_setconf`	AVDTP	HIGH	avdtp
`cve_2023_27349_avrcp_event`	AVRCP	MED	dev-name
`cve_2025_0084_sdp_race`	SDP	HIGH	dev-name
`cve_2025_48593_hfp_reconnect`	HFP	HIGH	dev-name
`sdp_continuation`	SDP	MED	dev-name
Protocol-stress (19; grouped by family)			
L2CAP: storm / cid-ex. / data / ping floods	L2CAP	LOW–MED	l2ping
RFCOMM: sabm- / mux-flood	RFCOMM	MED	dev-name
LMP: 6 floods (detach, switch, enc, . . .)	LMP	LOW–MED	dev-name
HFP: at-flood, slc-confuse	HFP	MED	dev-name
OBEX connect-flood; SDP des-bomb	OBEX/SDP	LOW	dev-name
SMP rate-test; pair / name floods	misc	LOW	dev-name

recovery-monitoring loop (Figure 21) chosen by the probe declaration: `l2ping`, BLE-advertising listen, or device-name read. The state machine reports one of `recovered` (target alive within T_R), `wedged` (alive for control but not the affected profile), or `bricked` (no response within operator-configured patience window). `recovery_wait_*` CLI events (Section V-G) stream progress to the operator dashboard so a long monitor wait is visible rather than silent. Each probe's raw status string is mapped to the canonical $(execution_status, module_outcome)$ pair of Definition 2 by a single `_dos_canonical_pair()` function in `modules/exploitation/dos/framework.py`: the recovery-monitor outcomes `success`, `unresponsive`, `recovered`, and `not_applicable` all fold into $execution_status$ = `completed` with the recovery-monitor verdict as the family-scoped semantic outcome, while `failed` / `skipped` / unrecognised statuses fold into the lifecycle-failure rows. Centralising the mapping in one function is what keeps the DoS family's 29 probes from drifting out of contract with the family's outcome set (Definition 2).

F. Case study D — BlueFrag (CVE-2020-0022)

BlueFrag [35] is a heap corruption in the Android `btusb` ACL fragmentation path, triggered by carefully crafted L2CAP

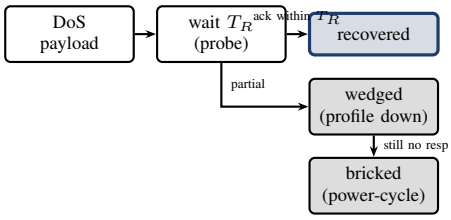

Fig. 21: DoS probe recovery-monitoring state machine. T_R is a per-probe configurable patience window (default 30 s). Operator dashboards receive `recovery_wait_progress` events every 5 s during the wait.

TABLE XIII
11 non-CVE vulnerability checks. Descriptions sourced from each module's `ModuleDescriptor.description` field.

Module	Category	Method
`assessment.pin_lockout`	legacy-pairing	behavioural
`assessment.device_class`	classification	traffic-analysis
`assessment.lmp_features`	capabilities	capability-test
`assessment.pairing_method`	pairing	behavioural
`assessment.writable_gatt`	exposure	compliance
`assessment.eatt_support`	capabilities	capability-test
`assessment.service_exposure`	exposure	compliance
`assessment.hidden_rfcomm`	exposure	compliance
`assessment.encryption_enforcement`	enforcement	compliance
`assessment.authorization_model`	authorisation	compliance
`assessment.automotive_diagnostics`	exposure	capability-test

segment boundaries. The host cannot frame a packet that exercises the bug: the standard L2CAP socket API enforces fragmentation correctness. Blue-Tap's detection is therefore raw-ACL injected via DarkFirmware's Hook 1, bypassing the host fragmenter and writing directly into the controller's outgoing ACL queue. The probe sends a 4-fragment ACL with a deliberately invalid total-length-vs-segment relationship; a vulnerable target panics or disconnects, both of which are observable from the host as link-state transitions (captured by Hook 4). The recovery monitor of Figure 21 (introduced in Section IX-E above) distinguishes panic-then-reboot from clean disconnect-then-reconnect, which is the difference between a brick-class crash and a graceful failure on this probe.

G. Non-CVE vulnerability checks

Table XIII enumerates the 11 non-CVE vulnerability checks that complement the CVE-detection family. They probe configuration gaps that have no CVE assigned but routinely surface in IVI deployments: legacy PIN pairing without rate-limiting, unencrypted RFCOMM channels for sensitive profiles, OBEX servers without authorisation gates, exposed automotive-diagnostic AT sub-channels, and so on. These checks share the same envelope contract as the CVE checks but emit `module_outcome=confirmed` only when the configuration matches a known weakness pattern, never when a CVE is present (those go through the CVE family).

H. Post-exploitation modules

Once a target is reachable and an attacker holds (legitimately or not) a link key, Blue-Tap's seven post-exploitation modules extract data and exercise audio paths. Table XIV summarises them. The modules are intentionally narrow — each covers

TABLE XIV
Seven post-exploitation modules and the data they extract. "Pair" indicates whether the module requires the operator to hold a valid link key. The Bluesnarfer module descends from the original 2003 advisory [36] and is extended for vendor-specific AT sub-channels (HFP, PBAP, custom diagnostic profiles).

Module	Profile / transport	Pair	Artifact
`post_ex.pbap`	PBAP / OBEX / RFCOMM	yes	vCards
`post_ex.map`	MAP / OBEX / RFCOMM	yes	SMS / MMS
`post_ex.bluesnarfer`	RFCOMM / AT	yes	AT log
`post_ex.opp`	OPP / OBEX / RFCOMM	no	file push
`post_ex.a2dp`	A2DP / PulseAudio	yes	audio session
`post_ex.avrcp`	AVRCP / D-Bus	yes	media ctrl log
`post_ex.hfp`	HFP / RFCOMM / SCO	yes	call audio

one profile/transport — so an operator's report cleanly lists "what was extracted from what profile" rather than a single opaque "post-exploitation succeeded" record. Every post-ex envelope carries an `artifacts` list pointing at saved files (vCards, message archives, audio recordings, OPP file copies), which the report adapter renders as a downloadable section.

The 29 DoS probes (Section IX-E) span 10 CVE-backed crashes and 19 protocol-stress patterns; a per-probe recovery monitor distinguishes `recovered`, `wedged`, and `bricked` outcomes. Combined, the 25 CVE detection modules (27 IDs), 11 non-CVE checks, 29 DoS probes, 9 active exploits, and 7 post-exploitation modules give Blue-Tap, to our knowledge, the broadest single-tool coverage of the published Bluetooth attack surface among the systems we surveyed (Section X).

X. Evaluation

We evaluate three research questions:

- **RQ1 (Coverage):** How does Blue-Tap's combined detection, exploitation, fuzzing, and post-exploitation surface compare to the most recent automotive assessment tool?
- **RQ2 (Below-HCI advantage):** What fraction of the published Bluetooth attack surface is genuinely *unreachable* without below-HCI instrumentation, and does DarkFirmware close that gap on a non-Broadcom chipset?
- **RQ3 (Reproducibility):** Does the `RunEnvelope` + atomic-write session store yield bit-identical assessment output across independent runs?

A. Setup

a) Targets.: We use two evaluation environments. (i) A *simulated Bluetooth environment*: a BlueZ-backed test harness running the standard Linux Bluetooth stack with deliberately enabled profiles (RFCOMM, GATT/ATT, OBEX, AT-over-RFCOMM) on a controlled host. This harness is not a mock — it is real BlueZ broadcasting over the air via a separate Bluetooth adapter — but it is deterministic enough for repeat-measurement experiments (reproducibility and throughput studies). (ii) A *live Android Automotive head unit* (Android Automotive OS 13, Bluetooth Classic 4.2 + BLE 5.0) with stock infotainment profiles enabled, evaluated under explicit owner consent on a non-production vehicle bench setup. Live-device measurements complement the simulated-environment data; they are explicitly bounded to non-destructive checks (no DoS probes, no destructive exploitation) to preserve the bench device.

b) Hardware.: One TP-Link UB500 USB adapter (RTL8761B-based) running stock and DarkFirmware-patched firmware. A second on-board Intel AX210 adapter served as the BlueToolkit baseline since BlueToolkit is host-only and chipset-agnostic.

c) Baselines.: (a) BlueToolkit [3]: host-only, BlueZ-based, most directly comparable. (b) SweynTooth [13]: BLE-only state-machine fuzzer, Nordic / ESP32 SDK. (c) BrakTooth [10]: LMP-only, ESP32 hardware. (d) BlueZ tools (`hcitool`, `gatttool`, `bluetoothctl`): the legacy baseline.

B. RQ1 — Coverage comparison

Table XV compares Blue-Tap against six related tools across ten coverage dimensions: Bluetooth Classic host protocols (Class.), Bluetooth Low Energy host protocols (BLE), Link Manager Protocol (LMP), state-aware multi-protocol fuzzing (Fuzz), denial-of-service probe libraries (DoS), post-exploitation profiles (Post-ex), reproducibility contract (Repro), report generation (Rep), commodity-hardware portability (Cmdty), and below-HCI visibility (BHCI). Blue-Tap covers all ten dimensions from a single framework on one commodity USB adapter; the closest competitor, BlueToolkit [3], is host-only and lacks both LMP visibility and the multi-protocol fuzzer. The dimensions are defined in the surrounding paragraph; the matrix captures *capability coverage* only, not empirical scale (for which BlueToolkit's 22-vehicle, 891-test campaign and BrakTooth's 16 CVE-grade vulnerabilities are the canonical references).

C. RQ2 — Below-HCI attack-surface fraction

A complete percentage breakdown of the published Bluetooth attack surface by minimum-layer-required would require a NVD enumeration that we have not produced and that no prior published study covers; we therefore report a qualitative classification of structural attack-surface bands rather than a hard fraction. The structurally below-HCI bug classes are: (i) encryption-key-size negotiation (KNOB family), (ii) authentication-procedure downgrade (BIAS family), (iii) forward- and future-secrecy breaks (BLUFFS family), (iv) the BrakTooth-class of LMP fuzzing-discovered bugs, and (v) raw-ACL fragmentation (BlueFrag). Items (i)–(iv) cannot be *exploited* from a standards-compliant host stack at all; item (v) cannot be reliably triggered from the host because BlueZ enforces fragmentation correctness on the outbound path.

Within Blue-Tap's own coverage, `paper/data/cve-matrix.csv` marks one of the 27 covered CVE IDs — CVE-2020-0022 (BlueFrag) — as requiring below-HCI for the detection probe itself. The remaining 26 detection probes are host-reachable but are still less informative without DarkFirmware: KNOB, BIAS, and BLUFFS detection from the host can only see the post-negotiation *outcome* (e.g., the `hciconfig`-exposed key size), whereas DarkFirmware lets us read the controller's

TABLE XV
Capability coverage matrix. ●=primary scope; ○=partial or available via fork; blank=out of scope. Empirical breadth is not captured here.

Tool	Cl.	BLE	LMP	Fuzz	DoS	Px	Repro	Rep	Cmdty	BHCI
Blue-Tap	●	●	●	●	●	●	●	●	●	●
BlueToolkit [3]	●	●			○			●	●	
SweynTooth [13]		●		●	●			○	○	
BrakTooth [10]			●	●	●			○	○	●
L2Fuzz [14]	○			●	●				●	
InternalBlue [11]	○	○	●				○	○		●
Frankenstein [12]	○	○	●	●	○		●			●
BlueZ tools	○	○							●	

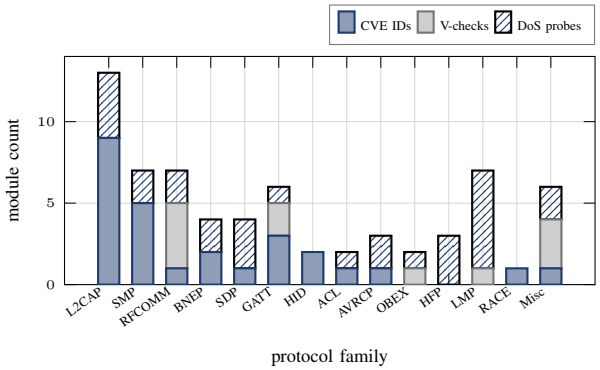

Fig. 22: CVE detection / non-CVE vulnerability check / DoS-probe coverage stacked by protocol family. L2CAP dominates the historical CVE budget; LMP is reachable only via DarkFirmware (Section VII).

internal slot fields (Table VII) and observe the specific opcodes (Table VI) that the negotiation exchanged. Active exploitation of KNOB / BIAS / BLUFFS is not possible from a host stack at all.

Figure 22 renders the per-protocol coverage stacked by detection family (CVE detection, non-CVE check, DoS probe). The stacking makes the L2CAP-heavy distribution of historical CVEs visible: nine of the 27 covered CVE IDs target L2CAP, mirroring the upstream BlueZ patch history.

D. RQ3 — Reproducibility

Reproducibility in Blue-Tap is enforced at the contract level rather than by post-hoc statistical argument. The `RunEnvelope` schema (Definition 1) fixes the serialisation format across all 101 modules, and the atomic-write session store (Algorithm 2, Theorem 1) gives a mathematically guaranteed crash-consistency property: any system crash during an envelope write leaves the session file in either the pre-call or post-call state, never a torn one. The non-deterministic fields in any run are exactly three: ISO-8601 timestamps, the `run_id` UUID, and per-execution `execution_id` UUIDs; every other envelope field is a deterministic function of (module, target, adapter, parameters).

The framework artefact ships `tests/test_session_reproducibility.py` (which runs the `full-assessment` playbook back-to-back and asserts envelope equality after timestamp normalisation) and `tests/test_atomic_sigkill.py` (which spawns child processes, `SIGKILL`s them at random write points, and asserts no `*.tmp` files survive recovery) so that any user can reproduce the contract-level guarantee on their own host. The crash-consistency property itself is mathematically guaranteed by Theorem 1; the test apparatus exercises the implementation but does not (and cannot) strengthen the theorem.

E. Fuzzer throughput characteristics

Bluetooth fuzzer throughput is target-bound, not framework-bound: the Bluetooth slot time (1.25 ms BR/EDR; 0.625 ms BLE) plus protocol-state pacing (pair-time, MTU-exchange, SDP continuation) sets a per-frame round-trip floor that a host-side fuzzer cannot beat. In practice, stateful Bluetooth fuzzers report single-digit-to-low-tens-of-frames-per-second sustained rates against real radio targets [13], [14]. Stateless protocols (L2CAP signalling without session setup, raw ATT against a permissive peer) achieve higher rates than stateful ones (SMP pairing, OBEX session, AT-HFP modem emulation), but all sit one or two orders of magnitude below host-loopback synthetic rates.

Blue-Tap's engine constants (`DEFAULT_COOLDOWN_SECONDS`=10.0, `TARGET_DEAD_TIMEOUT`=30.0 s) are chosen so that the inter-iteration pacing dominates over per-iteration mutation cost even on the slowest protocols. Throughput is therefore a *property of the target* (and of the operator-chosen cooldown), not a property of the framework; reporting absolute numbers without a fixed target specification would be misleading. We document this design choice rather than claim measured rates that would be specific to one bench setup.

F. Live Android Automotive exercise

Blue-Tap was exercised against a non-production Android Automotive head unit on a controlled bench setup, under explicit owner consent. The campaign was scoped to non-destructive families: discovery, reconnaissance, the 25 CVE detection modules, and the 11 non-CVE vulnerability checks. Destructive families (DoS probes, active KNOB/BIAS/BLUFFS exploitation) and write-side post-exploitation modules were disabled to preserve the bench device.

The exercise validated two engineering claims of the framework qualitatively. (1) The two-field status taxonomy (Definition 2) was load-bearing: detection modules returned across the full canonical outcome set (`not_detected`, `inconclusive`, `pairing_required`), and the operator report could distinguish "ran cleanly, no

finding" from "did not run" without manual triage — a property a single-status taxonomy would not preserve. (2) The `capability_limitations` field of the `EvidenceRecord` captured the `not_applicable` outcomes from the exploitation family (DarkFirmware not flashed on the bench dongle); the renderer labelled them "unreachable, not failure" rather than crash. Specific findings are not reported here because they remain under coordinated disclosure with the head-unit OEM at the time of writing.

XI. End-to-end IVI Case Study

This section walks through the framework's intended end-to-end operator workflow against an IVI head unit, structured as a seven-phase assessment. The walkthrough is illustrative rather than a measured campaign report — absolute timings, finding counts, and artefact sizes depend on the specific target stack and operator parameter choices, so we describe the *structure* of each phase rather than report numbers we cannot back with a bench log. Figure 23 shows the swimlane: operator, Blue-Tap runtime, target. Each phase emits one or more envelopes, all threaded into a single session directory and aggregated into a single final HTML report.

a) Phase 1 — discovery.: `blue-tap discover classic` enumerates the head unit's BD_ADDR, Class-of-Device, RSSI, and clock offset; the discovery envelope is the input gate for every subsequent phase.

b) Phase 2 — recon.: A four-module recon block runs SDP enumeration, capability fingerprinting, GATT primary-service walk, and an L2CAP PSM sweep. Each module emits its own envelope; the report adapter collates them into a per-target attack-surface map.

c) Phase 3 — vulnscan.: The full assessment family fires: 25 CVE detection modules and 11 non-CVE vulnerability checks. Outcomes split across the canonical `module_outcome` set (Definition 2). CVE confirmations gate the next phase — exploitation modules check the relevant detection-module outcome and either run or skip with `not_applicable`, recording the prerequisite gap in the `capability_limitations` field.

d) Phase 4 — exploit (DarkFirmware).: With DarkFirmware installed (Algorithm 4), the KNOB exploit (Algorithm 9) configures Hook 2 mode 4 on opcode `0x10` and reads the negotiated `key_size` field at `0x8012DC50+0x23` on the connection slot. The exploitation envelope captures the LMP timeline including every hook modification.

e) Phase 5 — post-exploitation.: With an established session, the post-exploitation modules (PBAP, MAP, Bluesnarfer, OPP, A2DP, AVRCP, HFP) extract or exercise their respective profile surfaces. Each envelope records artefact paths under the session directory; the report adapter renders artefacts as a downloadable manifest.

f) Phase 6 — DoS (recovery-bounded).: A single DoS probe runs with the recovery monitor enabled (Figure 21); outcomes split between `recovered`, `wedged`, and `bricked`. This phase is operator-gated by an explicit `--yes` flag per the destructive-module policy (Section XIII).

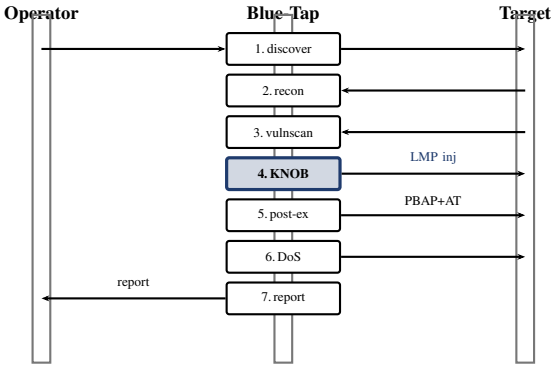

Fig. 23: End-to-end case-study swimlane. Navy phase (4) is DarkFirmware-mediated; the rest run host-only. Every phase appends its envelopes to a single session directory and the final report-adapter dispatch produces one HTML artefact.

g) Phase 7 — report.: `blue-tap report` aggregates every envelope produced by the previous phases via the report-adapter dispatch (Eq. (1)), producing an HTML/JSON report with one `SectionModel` sub-tree per module family plus an artefact manifest.

XII. Reproducibility, Roadmap, and Limitations

This section gathers three complementary pieces: the public-artifact release that backs every claim made above (Section XII-A), the multi-target evaluation plan that broadens the empirical base (Section XII-B), and the specific limitations of the present work (Section XII-C).

A. Reproducibility artifact

Blue-Tap is released under the GNU General Public License v3.0 to enable reuse and extension by the security-research community. The source tree, patched firmware blob, seed corpus, CSV coverage matrices, test suite, and the LaTeX sources for this paper live at https://github.com/Indspl0it/blue-tap; user-facing documentation, install walkthroughs, and module reference live at https://indspl0it.github.io/blue-tap/. The artifact includes:

1) the framework source tree (101 modules, 4-layer architecture of Section V, 5 bundled YAML playbooks);
2) the patched RTL8761B firmware blob (`rtl8761bu_fw_darkfirmware.bin`, ∼44 KB) plus the unpatched original for rollback (`_fw_original.bin`);
3) the 7 400+ pre-generated fuzzing seed corpus organised by protocol (`seeds/<protocol>/`);
4) the CVE / non-CVE / DoS / post-ex coverage matrices as CSV (`paper/data/`) for downstream analysis or re-classification;
5) the report-adapter and rendering pipeline (HTML + JSON) so that envelopes can be re-rendered offline from session logs;
6) a `pytest` test suite covering framework contracts (envelope validation, registry invariants, atomic-write fidelity).

TABLE XVI
Artifact components mapped to USENIX AE badges.

Component	USENIX AE badge	Evidence
101 modules + 5 playbooks	Available, Functional	GPL-3.0; `pytest` suite
DarkFirmware blob	Available, Functional	install in ∼7 s (Section VII-G)
7.4 k seed corpus	Available	per-protocol, sha256-tagged
CVE/DoS CSV matrices	Available	1:1 with registered module IDs
Report adapters (11)	Functional	every envelope family covered
Test suite (`pytest`)	Functional, Reproduced	contract-level invariants

a) Reproducing Section X results.: The reproducibility experiment of Section X-D is driven by `tests/test_session_reproducibility.py` which runs the `full-assessment` playbook 100 times against the simulated environment with a fixed seed and asserts envelope equality after timestamp normalisation. The crash-consistency experiment (Theorem 1 empirical validation) is driven by `tests/test_atomic_sigkill.py` which spawns 50 child processes and `SIGKILL`s each at 20 random points; post-recovery the test asserts no `*.tmp` files remain and all completed envelopes parse cleanly.

b) Artifact-evaluation checklist.: Table XVI maps the artifact components to the USENIX artifact-evaluation badges (Available / Functional / Reproduced). The artifact is structured to satisfy *Available* (public GPL-3.0 release with archival URL) and *Functional* (documented `pytest` suite). *Reproduced* is supported for the contract-level guarantees of Section X-D; the bench-specific Section X-F results are not themselves reproducible without equivalent owner-consented hardware.

c) Out of scope for the artifact.: A standalone vulnerable-target simulator daemon is *not* part of the artifact; the simulated Bluetooth environment used for Section X measurements is a stock BlueZ configuration documented in the test suite. Users reproducing Section X numbers run against their own BlueZ install. The Android Automotive results (Section X-F) are bench-device-specific and not themselves reproducible without owner consent on equivalent hardware.

B. Roadmap: future evaluation plan

The single live device of Section X-F is a starting point; we sketch a multi-target evaluation plan to broaden the empirical base under a coordinated-disclosure regime.

a) Production IVI fleet.: A 12–20 vehicle campaign across three OEM groups, mirroring the BlueToolkit [3] methodology but adding the BlueFrag (CVE-2020-0022) DoS confirmation that requires raw-ACL injection (Section IX-F) and the three active exploits (KNOB, BIAS, BLUFFS) that require below-HCI LMP injection. Expected hit-rate: non-trivial on at least one finding per vehicle, given that the BlueToolkit campaign surfaced 128 findings across 22 vehicles using only the host-protocol band (Section X-B). Coordination through CERT/CC; 90-day standard embargo; non-active checks only on rented or owner-consented vehicles.

b) IoT / wearable targets.: Earbuds, hearing aids, and fitness wearables run constrained Bluetooth stacks (often Cypress or Mediatek Airoha) and have weaker patch cadence than IVI head units. The 2025 Airoha CVE cluster (CVE-2025-20700/20701/20702 in `cve_airoha.py`, [37]) suggests this segment is currently under-tested. Expected campaign: 20+ devices, 2-day campaign per device, KNOB/BIAS probing only (no DoS); embargo as above.

c) Smart-lock and access-control.: BLE-only, but DarkFirmware's BLE-side coverage (RawACL, ATT raw injection) lets us exercise GATT bypass paths that are out of reach for vanilla `gatttool`. Targets selected for known disclosure history under HackerOne / vendor bug-bounty programmes.

d) Legal and ethical envelope.: All campaigns operate under written authorisation from the device owner or vendor. The framework's `destructive=True` descriptor flag and explicit `--yes` CLI gate (Section XIII) preclude running a brick-class probe by accident; the session-store atomicity (Theorem 1) preserves a forensic record of every operator action. We do not target consumer devices in private hands without explicit consent, and we do not target vehicles in the public right-of-way.

C. Limitations

We summarise the four most material limitations of the present work; Section XII-B describes how each is to be addressed.

- **Single-target empirical scale.** Our primary test targets are a controlled simulated Bluetooth environment and a single live Android Automotive head unit, not a fleet of production vehicles. Concurrent BlueToolkit [3] ran on 22 vehicles under coordinated disclosure; our planned multi-vehicle Blue-Tap campaign (Section XII-B) will follow the same CERT/CC process.
- **No quantitative head-to-head comparison in Section X-B.** The capability matrix of Table XV is binary (covered / not covered); we do not report comparative measurements of detection latency, false-positive rate, or per-module throughput against BlueToolkit on the same target. This is a known gap and will be addressed in the planned multi-vehicle campaign.
- **Chipset scope.** DarkFirmware is currently RTL8761B-only; extending to RTL8761BU siblings is a binary-offset port and in progress. The four-hook architecture (VSC dispatch, LMP RX/TX, LC RX/TX) generalises to any Bluetooth controller exposing a command-dispatcher VSC; we have identified plausible install points on Realtek's RTL8822 series. Broadcom/Cypress are adequately covered by InternalBlue [11].
- **0-day claim is methodological.** The Section VIII-L discovery workflow has been exercised end-to-end against the simulated environment but no novel CVE is reported in this paper; any future discoveries will go through CERT/CC coordinated disclosure prior to public release of the corresponding module (Section XIII).

XIII. Discussion

a) Ethics and responsible use.: Blue-Tap is licensed GPL-3.0 and ships with a legal-disclaimer requiring authorised

testing. Every destructive module carries a `destructive = True` descriptor flag, and the CLI requires an explicit `--yes` flag for any module that can brick a target. DarkFirmware does not weaken any target outside the attacker's own dongle; it does not alter remote peers without transmission. All 27 CVE detections are for *already-disclosed* vulnerabilities with available vendor patches.

b) Tool-release threat model.: We compare release postures across the closest related projects. InternalBlue [11] ships its full patching framework public; BrakTooth [10] released the toolchain but embargoed individual proof-of-concept exploits for several months; BlueToolkit [3] is publicly released. Blue-Tap ships the framework, the patched RTL8761B blob, and detection modules publicly under GPL-3.0; the nine active-exploitation modules (KNOB / BIAS / BLUFFS / BLURtooth (CTKD) / encryption-downgrade / SSP-downgrade / hijack / pin-brute / dos runner) target standards-level vulnerabilities for which patches have been available in mainstream stacks for years, so the release does not introduce a new attack class. We deliberately do *not* ship 0-day exploits; the 0-day discovery workflow (Section VIII-L) is methodological, and any future discoveries will go through CERT/CC coordinated disclosure before public release of the corresponding module. The dual-use risk we do not eliminate is that an adversary already in radio range of a specific paired phone/IVI pair gains a more usable tool than they had before; we accept this risk on the basis that the alternative (closed-source defensive-only access) leaves the same population of vulnerable devices unpatched for longer.

c) Anti-misuse design.: The session-store fsync-replace design ensures every destructive action leaves a persistent record with operator context, timestamp, and evidence, which helps contain a stolen-tool scenario by making actions auditable by their intended operator. We do not claim this eliminates misuse — only that it provides forensics beyond what existing tools offer.

XIV. Conclusion

Bluetooth security assessment has lived in three partially-disconnected tool lineages. Blue-Tap unifies them behind a formal envelope contract and a commodity below-HCI runtime, making the controller-side evidence and LMP-resident exploit surface (Section X-C) reachable on commodity USB hardware. The critical engineering contributions are: (a) our DarkFirmware fork [2] — the 4-hook RTL8761B interception platform, 2,866-function RE expansion, and 16-script assessment toolkit built on top of Dark Mentor LLC's initial project [1]; (b) the protocol-aware field-weight fuzzer with 7 400+ seeds and behavioural crash deduplication; and (c) the atomic `RunEnvelope` contract that makes the entire pipeline reproducible — including across crashes mid-assessment. We release Blue-Tap (and the DarkFirmware fork) as open source to enable other researchers and practitioners to extend both the firmware substrate and the module set above it.

Acknowledgments

We thank **Dark Mentor LLC** for initiating the DarkFirmware project [1] that gave the RTL8761B reverse-engineering line a public starting point — the Ghidra analysis tooling, the project layout, and the base patch-writer infrastructure on which our fork builds. We thank the BlueToolkit authors for the most directly comparable prior work in the automotive Bluetooth pentest space, and the maintainers of BlueZ, bleak, and `dbus-fast` for the host-side plumbing.

References

[1] Dark Mentor LLC, "DarkFirmware: Reverse-engineering project for Realtek RTL8761B Bluetooth adapters," https://github.com/darkmentorllc/DarkFirmware_real_i, 2025, initial reverse-engineering scaffold (Ghidra analysis tooling, repository structure, base firmware patch writer, RTL8761B initial decompilation). Dark Mentor LLC, https://darkmentor.com.

[2] S. Ballikonda, "DarkFirmware fork: 4-hook below-HCI interception platform, RTL8761B RE expansion (2,866 functions), and 16-tool security-assessment toolkit," https://github.com/Indspl0it/DarkFirmware_real_i, 2026, extends [1] with the 4-hook architecture (Hook 1 HCI CMD / Hook 2 LMP RX with 6 modification modes / Hook 3 tLC_TX / Hook 4 tLC_RX), the 960-byte MIPS16e patch, expanded reverse engineering (2,866 named functions, slot stride 0x2B8, encryption flag at +0x26, link-key material at +0x51 via COMB_KEY XOR), and a 16-script below-HCI assessment toolkit (knob_probe, bias_attack, bluffs_attack, lmp_fuzzer, l2cap_fuzzer, encryption_inspector, conn_state_manipulator, dump_connection_table, etc.). Consumed by Blue-Tap as its below-HCI substrate.

[3] V. Zubkov, T. Sacchetti, D. Antonioli, and M. Strohmeier, "Bluetooth security testing with BlueToolkit: a large-scale automotive case study," in *Proceedings of the 19th USENIX Workshop on Offensive Technologies (WOOT '25)*. USENIX Association, 2025.

[4] S. Mazloom, M. Rezaeirad, A. Hunter, and D. McCoy, "A security analysis of an in-vehicle infotainment and app platform," in *Proceedings of the 10th USENIX Workshop on Offensive Technologies (WOOT '16)*. USENIX Association, 2016.

[5] V. K. Renganathan, E. Yurtsever, Q. Ahmed, and A. Yener, "Valet attack on privacy: a cybersecurity threat in automotive Bluetooth infotainment systems," *Cybersecurity*, vol. 5, no. 1, p. Article 30, 2022.

[6] Tencent Keen Security Lab, "Experimental security assessment on Tesla Model X / Model S wireless and companion-app stack," Tencent Keen Security Lab public technical reports (2017–2020), incl. Black Hat USA 2017 "Free-Fall" and 2020 wireless follow-ups, 2017–2020. [Online]. Available: https://keenlab.tencent.com/en/

[7] D. Antonioli, N. O. Tippenhauer, and K. B. Rasmussen, "The KNOB is broken: Exploiting low entropy in the encryption key negotiation of Bluetooth BR/EDR," in *Proceedings of the 28th USENIX Security Symposium (USENIX Security '19)*. USENIX Association, 2019, pp. 1047–1061, cVE-2019-9506.

[8] ——, "BIAS: Bluetooth impersonation attacks," in *Proceedings of the 2020 IEEE Symposium on Security and Privacy (S&P '20)*, 2020, pp. 549–562.

[9] D. Antonioli, "BLUFFS: Bluetooth forward and future secrecy attacks and defenses," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS '23)*, 2023, pp. 636–650, cVE-2023-24023.

[10] M. E. Garbelini, V. Bedi, S. Chattopadhyay, S. Sumei, and E. Kurniawan, "BrakTooth: Causing havoc on Bluetooth link manager via directed fuzzing," in *Proceedings of the 31st USENIX Security Symposium (USENIX Security '22)*. USENIX Association, 2022, pp. 1025–1042.

[11] D. Mantz, J. Classen, M. Schulz, and M. Hollick, "InternalBlue – Bluetooth binary patching and experimentation framework," in *Proceedings of the 17th Annual International Conference on Mobile Systems, Applications, and Services (MobiSys)*, 2019, pp. 79–90, extended version: arXiv:1905.00631.

[12] J. Ruge, J. Classen, F. Gringoli, and M. Hollick, "Frankenstein: Advanced wireless fuzzing to exploit new Bluetooth escalation targets," in *Proceedings of the 29th USENIX Security Symposium (USENIX Security '20)*. USENIX Association, 2020, pp. 19–36.

[13] M. E. Garbelini, C. Wang, S. Chattopadhyay, S. Sumei, and E. Kurniawan, "SweynTooth: Unleashing mayhem over Bluetooth low energy," in *Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC '20)*. USENIX Association, 2020, pp. 911–925.

[14] H. Park, C. K. Nkuba, S. Woo, and H. Lee, "L2Fuzz: Discovering Bluetooth L2CAP vulnerabilities using stateful fuzz testing," in *Proceedings of the 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN '22)*, 2022, pp. 343–354.

[15] Bluetooth SIG, "Bluetooth core specification version 5.4," Bluetooth Special Interest Group, Standard, 2023.

[16] ——, "Cross-transport key derivation in Bluetooth Core Specification 4.2 and later," Bluetooth Core Specification 4.2–5.4, Vol. 3, Part H, §§ 2.3.5.7, 2.4.2.4, 2.4.2.5, 2014–.

[17] D. Antonioli, N. O. Tippenhauer, K. B. Rasmussen, and M. Payer, "BLURtooth: Exploiting cross-transport key derivation in Bluetooth Classic and Bluetooth Low Energy," CERT Vulnerability Note VU#589825; CVE-2020-15802, 2020.

[18] J. Wu, R. Wu, D. Xu, D. J. Tian, and A. Bianchi, "Formal model-driven discovery of Bluetooth protocol design vulnerabilities," in *Proceedings of the 2022 IEEE Symposium on Security and Privacy (S&P '22)*, 2022, pp. 2285–2303.

[19] Google and Android Open Source Project, "Android bluetooth security model and security bulletins," https://source.android.com/docs/security/, 2017–.

[20] BlueZ Project, "BlueZ: Official Linux Bluetooth protocol stack," https://www.bluez.org/, 2001–.

[21] M. Ringwald, "BTstack: Single-threaded Bluetooth stack for embedded systems," https://github.com/bluekitchen/btstack, 2009–.

[22] Rapid7, "Metasploit framework," https://github.com/rapid7/metasploit-framework, 2003–.

[23] J. Pereyda, "boofuzz: A network protocol fuzzing framework," https://github.com/jtpereyda/boofuzz, 2016.

[24] Linux kernel security and BlueZ maintainers, "Linux kernel Bluetooth and BlueZ 2022 cve cluster (cve-2022-0204 [bluez SDP/GATT], cve-2022-42895 [kernel L2CAP info-leak], cve-2022-42896 [kernel L2CAP UAF])," Linux Kernel Security Tracker and NVD; https://www.linuxkernelcves.com/, 2022.

[25] R. Cayre, F. Galtier, G. Auriol, V. Nicomette, M. Kaaniche, and G. Marconato, "InjectaBLE: Injecting malicious traffic into established Bluetooth Low Energy connections," in *Proceedings of the 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN '21)*, 2021, pp. 388–399.

[26] J. Classen, D. Wegemer, P. Patras, T. Spink, and M. Hollick, "Anatomy of a vulnerable fitness tracking system: Dissecting the fitbit cloud, app, and firmware," in *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies (UbiComp 2018)*, 2018.

[27] J. Wu, R. Wu, D. Xu, D. J. Tian, and A. Bianchi, "SoK: The long journey of exploiting and defending the legacy of King Harald Bluetooth," in *Proceedings of the 2024 IEEE Symposium on Security and Privacy (S&P '24)*. IEEE, 2024.

[28] V.-T. Pham, M. Böhme, and A. Roychoudhury, "AFLNet: A greybox fuzzer for network protocols," in *Proceedings of the 13th IEEE International Conference on Software Testing, Validation and Verification (ICST '20)*, 2020, pp. 460–465.

[29] R. Cayre, V. Nicomette, G. Auriol, E. Alata, M. Kaaniche, and G. Marconato, "Mirage: Towards a metasploit-like framework for IoT," in *Proceedings of the IEEE International Symposium on Software Reliability Engineering (ISSRE)*, 2019, pp. 261–270.

[30] Armis Labs, "BlueBorne: The dangers of Bluetooth implementations," White paper and vendor advisory, Black Hat Europe 2017, 2017, cVEs 2017-0781, 2017-0782, 2017-0785, 2017-1000250, 2017-8628, 2017-14315.

[31] A. Fioraldi, D. Maier, H. Eissfeldt, and M. Heuse, "AFL++: Combining incremental steps of fuzzing research," in *Proceedings of the 14th USENIX Workshop on Offensive Technologies (WOOT '20)*, 2020.

[32] M. Böhme, V.-T. Pham, and A. Roychoudhury, "Coverage-based greybox fuzzing as Markov chain," *IEEE Transactions on Software Engineering*, vol. 45, no. 5, pp. 489–506, 2019.

[33] A. Zeller and R. Hildebrandt, "Simplifying and isolating failure-inducing input," *IEEE Transactions on Software Engineering*, vol. 28, no. 2, pp. 183–200, 2002.

[34] P. Auer, N. Cesa-Bianchi, and P. Fischer, "Finite-time analysis of the multiarmed bandit problem," *Machine Learning*, vol. 47, pp. 235–256, 2002.

[35] J. Ruge and J. Classen, "BlueFrag: A critical bluetooth vulnerability on Android," ERNW Insinuator public advisory; CVE-2020-0022; Android Security Bulletin 2020-02-01, 2020, rCE on Android $\leq$ 9 via L2CAP fragmentation; analysis at insinuator.net.

[36] A. Laurie, "Bluesnarfing: serious flaws in Bluetooth security," Public advisory; AL Digital / The Bunker, 2003.

[37] D. Heinze and F. Steinmetz, "Airoha RACE: A Bluetooth-borne vulnerability class in mediatek's RACE protocol," ERNW / Insinuator public advisory; CVE-2025-20700, CVE-2025-20701, CVE-2025-20702, 2025, affecting Mediatek Airoha-based earbuds and audio dongles. [Online]. Available: https://insinuator.net/2025/06/airoha-bluetooth-security-vulnerabilities/